# 河南大學

# 本科毕业论文（设计）

## 基于 Spring Cloud 的动态负载均衡选课系统设计与实现


作 者 姓 名：石啸林

作 者 学 号：2198020031

所 在 学 院：濮阳工学院

所 学 专 业：软件工程

导 师 姓 名：李颜兴、孙媛

导 师 职 称：助教、副教授

导 师 单 位：河南大学/濮阳工学院


2025 年 04 月 25 日

# 摘 要

本文设计并实现了一个基于微服务架构的动态负载均衡选课系统，重点解决了传统选课系统在高并发场景下的稳定性、安全性和用户体验问题。传统选课系统在高并发场景下面临三大核心问题：一是系统稳定性不足，选课高峰期易出现响应延迟或崩溃；二是数据一致性难以保障，重复选课和超卖现象频发；三是安全性薄弱，缺乏统一的鉴权和细粒度权限控制。

针对上述问题，本文采用 Spring Cloud 微服务架构，结合 Docker 容器化和 Kubernetes 集群管理技术，实现动态扩缩容和自动化运维，有效应对选课高峰期的流量冲击，并利用 Horizontal Pod Autoscaler（HPA）动态调整资源分配，确保系统在高并发下仍能保持稳定运行。选课模块引入 Redis 缓存和 Redission 分布式锁，结合 RabbitMQ 消息队列实现异步处理，有效防止重复选课和系统过载，同时通过幂等性机制保障数据一致性。使用 Sa-Token 保证用户信息在容器之间安全迅速传递，更为以后分布式扩展打下基础。

该系统在高并发环境下表现优异，能够稳定支持大规模选课需求，为高校教务管理提供了高效、安全、可靠的解决方案。

**关键词：**微服务；动态扩容；选课系统

# ABSTRACT

This paper designs and implements a dynamic load balancing course selection system based on microservice architecture, focusing on solving the stability, security and user experience problems of the traditional course selection system in high concurrency scenarios. The traditional course selection system faces three core problems in high concurrency scenarios: first, the system stability is insufficient, and response delays or crashes are easy to occur during the peak period of course selection; second, data consistency is difficult to guarantee, and the phenomenon of duplicate selection and overselling occurs frequently; and third, security is weak, and there is a lack of unified authentication and fine-grained privilege control.

To address the above issues, this paper adopts Spring Cloud microservice architecture, combined with Docker containerization and Kubernetes cluster management technology, to achieve dynamic expansion and contraction and automated operation and maintenance, to effectively cope with the traffic impact of the peak period of class selection, and uses Horizontal Pod Autoscaler (HPA) to dynamically adjust the allocation of resources, to ensure that the system can still maintain a high level of concurrency. It also utilizes HPA to dynamically adjust resource allocation to ensure that the system can still maintain stable operation under high concurrency. The course selection module introduces Redis cache and Redission distributed lock, combines with RabbitMQ message queue to realize asynchronous processing, effectively prevents duplicate course selection and system overload, and at the same time guarantees data consistency through the idempotency mechanism. The use of Sa-Token ensures the safe and rapid transfer of user information between containers and lays the foundation for future distributed expansion.

The system performs well in a highly concurrent environment, can stably support large-scale course selection requirements, and provides an efficient, safe and reliable solution for university academic affairs management.

**Keywords:** Microservices; Dynamic scaling; Course Selection System

# 目　录

摘　要 I
ABSTRACT II
第 1 章　绪论 1
1.1 课题研究背景及意义 1
1.2 国内外研究现状 2
1.3 论文的主要研究内容 3
1.4 论文的组织结构 5
1.5 本章小结 6
第 2 章 系统开发相关技术分析 7
2.1 相关技术 7
2.1.1 Spring Cloud 微服务架构 7
2.1.2 Docker 与 Kubernetes (K8S) 集群管理 7
2.1.3 动态负载均衡与自动扩缩容 7
2.1.4 Redis 缓存与 Redission 分布式锁 8
2.1.5 消息队列与异步处理 8
2.1.6 前端 UI 开发 8
2.1.7 系统安全与统一鉴权 9
2.2 开发环境 9
2.2.1 开发工具 9
2.2.2 运行环境 11
2.2.3 安全与监控 11
2.3 本章小结 12
第 3 章 可行性与需求分析 13
3.1 可行性分析 13
3.1.1 技术可行性 13
3.1.2 用户操作体验可行性 13
3.1.3 开发成本可行性 14

3.2 功能需求分析.....14
3.2.1 功能描述.....14
3.2.2 管理员用例图.....15
3.2.3 用户综合用例图.....15
3.3 本章小结.....16
第 4 章 系统概要设计.....17
4.1 系统架构设计.....17
4.2 总体结构设计.....17
4.2.1 用户服务层.....17
4.2.2 核心业务层.....18
4.2.3 数据支撑层.....19
4.3 功能模块设计.....19
4.3.1 负载均衡动态扩缩模块.....19
4.3.2 选课模块.....20
4.3.3 系统管理模块.....20
4.3.4 消息通知模块.....21
4.3.5 前端 UI 模块.....21
4.3.6 网关与微服务模块.....21
4.3.7 系统安全模块.....21
4.4 数据库设计.....22
4.4.1 概念设计.....22
4.4.2 逻辑设计.....23
4.5 本章小结.....26
第 5 章 系统详细设计与实现.....28
5.1 负载均衡动态扩缩模块.....28
5.1.1 Kubernetes 进行集群管理和资源调度.....28
5.2 选课模块的实现.....29
5.2.2 选课操作页面.....31
5.2.2 选课完成页面.....32

5.3 系统管理模块的实现.....33
5.3.1 学生管理页面.....33
5.3.2 课程管理页面.....33
5.3.3 教师管理页面.....34
5.4 消息通知模块的实现.....34
5.4.1 消息通知页面.....35
5.5 网关与微服务模块的实现.....36
5.6 系统安全模块的实现.....37
5.7 本章小结.....39
第 6 章 系统测试.....41
6.1 测试设计.....41
6.2 主要功能测试.....41
6.2.1 系统安全功能测试.....41
6.2.2 选课模块功能测试.....42
6.2.3 系统管理模块功能测试.....43
6.2.4 消息通知模块功能测试.....44
6.3 性能压力测试.....45
6.4 本章小结.....46
第 7 章 总结与展望.....47
7.1 总结.....47
7.2 展望.....47
参考文献.....49
致谢.....51

# 第 1 章　绪论

在高等教育数字化转型的背景下，选课系统作为支撑教学管理的关键基础设施，其性能优化和稳定性保障面临着新的技术挑战。特别是在春季/秋季学期选课高峰期，系统需要同时应对大规模并发访问、课程资源动态分配以及公平性保障等多重需求。传统的选课系统在高并发访问、数据安全和用户体验等方面存在诸多不足，尤其是在学期初选课高峰期，系统容易出现响应缓慢甚至崩溃的情况，严重影响了学生的选课体验。为此，本课题旨在构建一个基于 Spring Cloud 微服务架构的高效、稳定、安全的选课系统，解决现有系统中的关键问题。

## 1.1 课题研究背景及意义

在高校和教育机构中，选课系统是教学管理系统的重要功能之一，尤其是在开学初，大量学生同时在线选课，可能导致系统过载、响应缓慢甚至崩溃。传统的选课系统往往采用固定的服务器部署方案，难以根据实时流量调整资源分配，这不仅影响了学生的选课体验，也增加了系统的运营成本。为了解决这些问题，本文引入 Spring Cloud 框架和动态负载均衡技术，有效提高选课系统的稳定性和性能，确保在高并发情况下的良好用户体验[1]。通过实现动态负载均衡，系统能够根据当前的访问压力自动调整各节点的负载，使得计算资源得到更合理的利用，避免资源闲置或过度使用，从而降低运营成本。

本研究采用 Spring Cloud 微服务框架作为技术基础，该框架通过组件化设计整合了服务发现、配置中心、API 网关等核心功能模块，有效解决了传统单体架构在选课系统场景下的扩展性瓶颈问题。将 Spring Cloud 应用于选课系统，不仅有助于推动教育信息化的发展，还为其他领域提供了参考案例，促进了云计算、微服务等新技术的应用与创新。基于 Spring Cloud 微服务架构实现的选课系统，通过服务解耦和独立部署的特性，不仅实现了功能模块的高内聚低耦合，还显著提升了系统的可扩展性（支持横向扩展）和可维护性（单个服务独立更新）。随着学校课程设置的变化或新需求的出现，系统可以快速适应，添加新的服务或修改现有服务，而无需对整个系统进行大规模改动，增强了系统的可扩展性和灵活性。

在数字化转型浪潮推动下，教育机构正加速将云计算、大数据等新一代信息技术深度融入教学管理全流程，以构建智能化、个性化的现代教育服务体系。本课题的研究成果符合这一发展趋势，为教育行业的数字化转型提供了有力支持，同时也顺应了国家关于推进智慧校园建设的相关政策。通过技术创新和优化资源配置，本课题旨在构建一个高效、稳定、安全的选课系统，不仅解决了实际问题，也为未来的教育信息化发展奠定了坚实的基础。

## 1.2 国内外研究现状

国内外关于基于 Spring Cloud 的动态负载均衡选课系统的研究，正逐渐从理论探索转向实际应用，尤其是在高校和教育机构中，这一趋势尤为明显。

清华大学采用 Spring Cloud 微服务架构重构了其选课系统，并利用 Nacos 进行服务注册与发现。该系统能够根据实时流量自动调整各节点的负载，显著提高了选课效率和用户体验。复旦大学在其智慧校园项目中采用了 Spring Cloud 技术栈，将选课系统与其他教育管理信息系统整合在一起。借助 Kubernetes 容器编排工具，系统实现了服务的自动伸缩和负载均衡，不仅提升了系统的性能，还简化了运维工作[2]。此外，系统还集成了 AI 驱动的预测模型，帮助学生更有效地选择课程，体现了智能化管理的优势。

斯坦福大学的在线学习平台（Stanford Online）也采用了类似的技术方案。平台使用 Spring Cloud 框架来管理多个微服务，利用 Prometheus 监控系统和 Grafana 可视化工具实时监测服务状态，并通过 Istio 服务网格实现精细化的流量管理和负载均衡。平台还应用了机器学习算法为学生提供个性化课程推荐，进一步提升了用户体验和服务质量。麻省理工学院（MIT）开放课程（MIT OpenCourseWare, OCW）在扩展其选课功能时，引入了 Spring Cloud 和 Kubernetes 技术。平台可以根据全球用户的访问情况动态调整服务器资源，保证了在高峰时段的服务稳定性。同时，OCW 还利用大数据分析为用户提供定制化的学习体验，推动了开放教育资源（OER）的发展和共享。

综上所述，国内外基于 Spring Cloud 的动态负载均衡选课系统研究虽已取得显著成果，但仍存在诸多技术挑战亟待解决。现有研究通过微服务架构、容器化部署和智能算法，有效提升了系统的可用性和智能化水平，但在高并发稳定性、

跨学科推荐准确性、数据稀疏性处理等方面仍存在明显不足。这些未解决的问题为后续研究指明了方向，特别是在算法优化、异构数据处理和多云协同等关键领域仍需重点突破。本文研究应着力攻克这些技术瓶颈，以推动教育信息化向更智能、更稳定的方向发展。

## 1.3 论文的主要研究内容

本研究聚焦高校选课系统的三大核心需求：高并发处理能力、事务一致性和系统安全性，通过构建基于 Spring Cloud 的微服务架构，实现了包含动态负载均衡、多层级安全防护等关键技术方案。为了应对高并发访问和保证系统的稳定性，负载均衡动态扩缩模块采用了 Docker 容器化部署，并结合 Kubernetes（K8S）进行集群管理和资源调度，支持自动扩展、滚动更新和故障恢复[3]。Horizontal Pod Autoscaler (HPA) 根据设定的指标自动调整 Pod 的数量，优化资源利用[4][5]。Redission 分布式锁和 Redis 缓存解决了高并发场景下的数据一致性和性能问题，确保选课等操作的顺利进行[6]。

针对选课高峰期的高并发问题，选课模块专注于提升系统的稳定性和响应速度。系统使用 Spring Cloud 微服务架构，将选课服务独立部署，减轻主系统的负担。为了应对大量并发请求，模块引入了 Redis 缓存，存储热门课程信息，减少数据库查询压力。Redission 分布式锁确保每个学生在同一时间只能发起一次选课请求，防止重复选课。消息队列（如 RabbitMQ）用于排队异步处理，将选课请求按顺序处理，避免系统过载[7]。同时，系统实现了幂等性机制，确保同一选课请求多次提交只被处理一次，防止数据异常[8]。

系统管理模块涵盖了用户管理、学生管理、课程管理、教师管理多个子模块，提供全面的教育管理功能。用户管理模块基于 RBAC 模型，为不同角色（如管理员、教师、学生）分配权限，确保每个角色只能访问与其职责相关的功能和数据。学生管理模块维护学生的学籍信息[9]。课程管理模块允许教师创建和编辑课程信息，管理员或教务处审核课程。教师管理模块维护教师的基本信息和教学安排，支持教师查看自己的课程安排和学生名单。

消息通知模块负责向用户发送系统消息和选课结果通知，确保信息的及时传递。选课期间，系统会通过消息队列 RabbitMQ 异步发送选课结果通知，告知学

生选课是否成功。如果选课失败，系统会提示学生选课失败及原因。此外，系统还会发送课程调整、停课、补课等通知，确保学生及时了解课程变动情况。通过消息队列的异步处理机制，系统能够高效地处理大量通知任务，确保通知的及时性和准确性。

前端 UI 模块采用 React 框架和 Ant Design 组件库，构建了一个现代化、响应式的用户界面[10]。系统采用了统一的设计风格，确保各个页面的一致性和用户体验的流畅性。前端与后端通过 RESTful API 进行通信，使用 Axios 库进行 HTTP 请求，确保数据传输的安全性和可靠性。通过这些技术手段，前端 UI 模块为用户提供了一个友好、直观的操作界面，提升了整体的用户体验。

网关与微服务模块基于 Spring Cloud Gateway 实现统一鉴权和路由管理，确保系统的安全性和灵活性。Nacos 用于微服务的注册和发现，确保服务之间的可靠通信。Open Feign 作为服务通信的方式。系统采用分布式架构，独立部署不同服务模块，分散负载，提升可扩展性和稳定性。通过这些技术手段，系统能够在高并发和复杂环境下保持高效、长久运行，确保用户体验的流畅性。

系统安全模块是确保选课系统的数据安全、用户隐私和整体稳定性的关键部分。Nginx 作为前端网关，不仅提供高效的反向代理功能，还通过域名隐藏了后端服务的真实 IP 地址，增加了系统的安全性[11]。所有外部请求首先经过 Nginx 进行处理，使用 SSL/TLS 加密和 HTTPS 协议确保数据传输的安全性，防止中间人攻击和数据泄露[12]。为了实现统一鉴权和细粒度的权限管理，系统采用 JWT（JSON Web Token）与 Spring Gateway 搭配。用户登录时生成包含用户信息和权限的 JWT 令牌，每次请求附带在 HTTP 头中，Spring Gateway 负责验证令牌的有效性，确保只有经过认证的用户才能访问系统资源。本系统采用改进的 RBAC 权限模型，通过角色分级和实现细粒度的访问控制，确保教学数据的安全隔离与合规使用。本系统构建了纵深防御安全体系，通过身份认证、访问控制、数据加密和操作审计四重防护机制，形成完整的网络安全闭环，在确保业务流畅性的同时实现敏感数据的全生命周期保护。

通过以上模块的协同工作，本文不仅解决了现有选课系统中的关键问题，还提升了系统的整体性能、稳定性和安全性，为高校和教育机构提供了更加高效、可靠的选课管理解决方案。

## 1.4 论文的组织结构

第一章绪论从教育信息化实践需求出发，阐明本研究解决的具体问题，通过对比分析现有研究成果的不足，提出基于微服务架构的选课系统优化方案，并概述论文各章节的技术实现路径。

第二章：系统开发相关技术分析，深入介绍了与系统开发相关技术及对各个开发环境进行分析，同时对本系统涉及的相关算法和技术、开发框架、设计模式以及构建系统所使用的关键技术进行介绍分析。

第三章从实际业务场景出发，首先分析了选课系统面临的核心痛点（如选课高峰期的系统崩溃、权限管理混乱等），然后针对性地提出了功能需求（学生选课、教师开课等基础功能）和非功能需求（系统响应速度、并发承载能力等），最终形成了一套完整的系统建设目标。

第四章：系统概要设计，本章围绕系统架构设计展开，首先从整体层面阐述技术选型与分层架构，明确用户服务层、核心业务层和数据支撑层的功能划分。随后详细分析各功能模块的设计方案，包括负载均衡动态扩缩、选课业务处理、系统管理等核心模块的实现逻辑。最后，完成了数据库的概念设计（E-R 图）和逻辑设计（关系模式转换），重点解决了课程-学生多对多关系，为系统实现提供了优化的数据存储方案。

第五章：系统详细设计与实现，重点包括：Kubernetes 集群的动态扩缩与监控、选课业务的高并发处理、系统管理的增删改查操作、消息通知的实时推送机制，以及网关路由与安全防护的实现。通过代码实现与逻辑分析，验证各模块的功能完整性与技术可行性。

第六章：系统测试，本章通过功能测试与性能测试，重点验证了选课高峰期系统核心功能模块的稳定性和并发处理能力。针对负载均衡、选课流程、系统管理、消息通知等核心功能设计测试用例，确保业务逻辑的正确性。同时通过阶梯式压力测试验证系统在高并发用户下的吞吐量和响应时间表现为性能调优提供数据支撑。

第七章：从技术实现和应用价值两个维度，总结了微服务架构在选课系统中的实践效果，并针对教育信息化发展趋势提出了更多的系统演进方向。

## 1.5 本章小结

本章基于高校信息化建设需求，首先阐述了选课系统优化的现实意义，继而通过文献分析指出现有研究的不足，重点介绍了本论文在微服务架构下的四大核心模块创新：智能选课引擎、动态权限管理系统、多通道消息通知服务及弹性扩缩容机制的设计与实现。最后梳理本论文各章论述核心并总结。

# 第 2 章 系统开发相关技术分析

## 2.1 相关技术

### 2.1.1 Spring Cloud 微服务架构

Spring Cloud 是一个为微服务架构设计的开发框架，集成了众多开源项目以简化微服务的开发与维护[13]。本课题采用 Spring Cloud 构建选课系统，通过其核心组件有效解决了分布式架构中的关键问题：Nacos 服务注册中心解决了微服务实例动态管理难题，实现服务的自动注册与发现；OpenFeign 通过声明式 REST 调用解决了微服务间复杂通信问题，简化了远程调用过程；Spring Cloud Gateway 则解决了 API 统一接入和安全管控问题，提供路由转发、负载均衡和权限校验等核心功能。这些技术协同工作，共同确保了系统的高可用性、可扩展性和安全性，为选课系统提供了技术支撑。

### 2.1.2 Docker 与 Kubernetes (K8S) 集群管理

本系统采用 Docker 容器封装微服务组件，通过 Kubernetes 编排引擎实现集群的自动化部署与弹性扩缩，具体包括 Pod 水平自动扩展（HPA）和滚动更新策略，确保选课高峰期服务的持续可用性。本系统采用 Docker 容器化技术，通过分层镜像构建（基础镜像+应用层+配置层）和卷挂载机制，实现了包括 JDK 环境、中间件配置和应用程序的三位一体标准化封装，确保从开发到生产环境的全链路一致性。Kubernetes 则提供了强大的容器编排能力，支持自动扩展、滚动更新和故障恢复，确保系统的高可用性和灵活性。通过 Horizontal Pod Autoscaler (HPA)，Kubernetes 可以根据预设的指标（如 CPU 使用率）自动调整 Pod 的数量，优化资源利用，确保系统在高并发情况下的稳定运行。

### 2.1.3 动态负载均衡与自动扩缩容

本系统采用 Nginx 作为前端静态资源服务器，通过缓存优化和 Gzip 压缩提升静态内容传输效率；同时基于 Spring Cloud Gateway 集成 Nacos 服务发现，实

现动态负载均衡（采用加权轮询算法），构建了分层流量治理体系。Horizontal Pod Autoscaler 是 Kubernetes 的一个重要特性，能够根据预设的指标（如 CPU 使用率或其他自定义指标）自动调整 Pod 的数量，优化资源利用。这使得系统能够在流量高峰期自动扩展，确保高并发情况下的稳定运行，同时在低谷期减少资源消耗，降低运营成本。

### 2.1.4 Redis 缓存与 Redission 分布式锁

为了提高系统的性能和数据一致性，本课题集成了 Redis 非关系型数据库和 Redission 分布式锁。Redis 缓存用于存储热门课程信息，减少直接访问数据库，提升系统的响应速度。Redission 分布式锁则确保每个学生在同一时间只能发起一次选课请求，防止重复选课，保证数据的一致性。Redis 作为分布式缓存中间件，通过其丰富的数据结构（String/Hash/Set 等）和原子操作特性，实现选课库存的精准控制与秒级更新，同时利用内存存储优势将课程查询延迟控制在 10ms 以内。Redission 则是一个基于 Redis 实现的分布式锁解决方案，能够确保在高并发场景下，每个学生在同一时间只能发起一次选课请求，防止重复选课。

### 2.1.5 消息队列与异步处理

为了处理选课高峰期的大量并发请求，本文引入了消息队列（Message Queue），用于排队异步处理选课请求。本系统采用 RabbitMQ 作为消息队列处理服务应用程序，其将选课请求按顺序处理，避免系统过载，同时确保每个请求都能得到及时处理。此外，消息队列还用于发送选课结果通知、课程调整通知等，确保信息的及时传递。通过消息队列流量削峰功能，系统可以将选课请求按顺序处理，避免系统过载，确保每个请求都能得到及时处理。为了防止同一选课请求多次提交导致数据异常，系统实现了幂等性机制。无论用户提交多少次相同的请求，系统只会处理一次，确保每次相同的请求取得相同的结果。

### 2.1.6 前端 UI 开发

前端 UI 模块采用 React 框架和 Ant Design 组件库[错误!未找到引用源。]，构建了一个现代化、响应式的用户界面。React 是一个流行的 JavaScript 库，支持组件化开

发，结合 TypeScript 类型约束，形成高内聚、低耦合的模块化开发体系。Ant Design 提供了丰富的 UI 组件，如表格、表单、按钮、对话框等，本文通过 React 作为前端交互界面，能够轻易的搭建美观、易用的学生选课界面。鉴于本文项目主要用于学生选课与教师发布课程，使用 Ant Design 的响应式模式，能够更好的适应移动端访问，自动适配不同比例的设备[14]。

### 2.1.7 系统安全与统一鉴权

系统安全模块是确保选课系统的数据安全、用户隐私和整体稳定性的关键部分。Nginx 作为前端网关，不仅提供高效的反向代理功能，还可以通过域名隐匿服务器 IP 地址。所有外部请求首先经过 Nginx 进行处理，使用 SSL/TLS 加密和 HTTPS 协议确保数据传输的安全性。同时，所有外部请求首先经过 Nginx 进行处理，确保流量的合理分发。

为了实现统一鉴权和细粒度的权限管理，系统采用 JWT（JSON Web Token）与 Spring Gateway 搭配。用户登录时生成包含用户信息和权限的 JWT 令牌，每次请求附带在 HTTP 头中，Spring Gateway 负责验证令牌的有效性，确保只有经过认证的用户才能访问系统资源。

通过以上技术手段，本课题不仅解决了现有选课系统中的关键问题，还提升了系统的整体性能、稳定性和安全性，为高校和教育机构提供了更加高效、可靠的选课管理解决方案。

## 2.2 开发环境

为了确保选课系统的高效开发、测试和部署，本课题构建了一个完善的开发环境，涵盖了开发工具、运行环境、依赖管理、持续集成与持续部署（CI/CD）等多个方面。以下是详细的开发环境配置：

### 2.2.1 开发工具

1、集成开发环境 (IDE)：

本系统开发采用 IntelliJ IDEA 作为核心 IDE，通过其深度集成的 Spring Cloud Alibaba 工具链（Nacos/Dubbo/Sentinel）、Kubernetes 插件和 Docker 支持，实现

了从微服务编码（含 JPA 智能提示）、容器化构建到云原生部署的全流程开发支持。

本系统前端开发采用 VS Code 作为核心 IDE，通过集成 ESLint 代码检查、React/Redux 插件和 Ant Design 组件库智能提示，配合 Live Server 实时预览功能，显著提升了界面开发效率和代码质量。

2、版本控制系统：

**Git**:采用 Git 进行版本控制，通过特性分支开发（feature branch）+ Pull Request 代码评审机制，结合 Git Flow 工作流规范，实现多人协作开发的高效管理，确保代码变更可追溯（commit message 规范）和版本可控（tag 标记发布版本）。

**Gitee**：作为国内主流的代码托管平台，Gitee（码云）提供了代码仓库、项目管理、持续集成等功能，方便团队协作和代码审查，同时在国内访问速度更快，符合本地化开发需求。

3、数据库管理工具：

Navicat Premium 15：用于设计和管理 MySQL 数据库，支持数据库建模、查询、备份和恢复等功能。

Another Redis Desktop Manager：用于管理和监控 Redis 数据库，支持键值操作、数据可视化和性能监控。

4、容器化工具：

Docker Desktop：本系统开发采用 Docker Desktop 作为本地容器化平台，通过多阶段构建（multi-stage build）和 Compose 编排技术，实现开发环境（带调试工具）、测试环境（集成测试套件）与生产环境镜像的统一化管理和无缝迁移，消除"在我机器上能跑"的环境差异问题。

5、Kubernetes 开发工具：

Minikube：用于本地搭建 Kubernetes 集群，方便开发者在本地环境中进行 Kubernetes 的开发和测试。

kubectl：Kubernetes 命令行工具，用于快速切换 Kubernetes 上下文和命名空间，提升开发效率。

### 2.2.2 运行环境

1、Java 运行环境：

JDK 11：作为 Java 应用程序的运行环境，JDK 11 提供了长期支持（LTS），确保系统的稳定性和安全性。

Maven：作为依赖管理工具，Maven 用于管理项目的依赖库和构建过程，支持自动化构建、测试和部署。

2、Node.js 运行环境：

Node.js 16：作为前端开发的运行环境，Node.js 提供了对 React、Ant Design 等前端框架的支持，确保前端应用的高效开发和运行。

npm：本系统前端工程采用 npm 进行依赖管理，通过 package.json 的语义化版本控制，实现依赖库的自动化安装、版本锁定（package-lock.json）及预发布质量管控。

3、数据库：

MySQL 8.0：作为关系型数据库，MySQL 8.0 提供了高性能、高可靠性和丰富的功能，适合存储选课系统的用户信息、课程信息、选课记录等数据。

Redis 6.0：作为内存数据库，Redis 6.0 用于缓存热门课程信息，用户可通过内存访问数据，大大降低数据库访问压力，同时提升响应速度性能。

4、消息队列：

RabbitMQ 3.8：作为消息队列系统，RabbitMQ 3.8 支持多种消息协议和持久化机制，用于处理选课高峰期的大量并发请求，确保选课请求按顺序处理，避免系统过载。

### 2.2.3 安全与监控

Nginx 作为反向代理和负载均衡器，Nginx 不仅提供高效的流量分发功能，还通过 SSL/TLS 加密和 HTTPS 协议对数据加密。Nginx 还支持 HSTS（HTTP Strict Transport Security），强制浏览器始终使用 HTTPS 访问，进一步提升安全性。

SaToken 与 Spring Gateway 统一鉴权。为了实现统一鉴权和细粒度的权限管

理，系统采用 SaToken 与 Spring Gateway 搭配。用户登录时生成包含用户信息和权限的令牌，每次请求附带在 HTTP 头中，Spring Gateway 负责验证令牌的有效性，确保只有经过认证的用户才能访问系统资源。基于 RBAC 模型，系统为不同角色（如管理员、教师、学生）分配不同的权限，确保每个用户只能访问与其职责相关的功能和数据。

## 2.3 本章小结

本系统通过科学配置开发环境，构建了规范化的研发流程体系，为项目顺利实施提供了有力保障。在开发过程中，团队建立了标准化的协作机制，确保各环节紧密衔接；通过合理的资源调配，实现了开发效率的显著提升；同时建立了完善的质量保障体系，为系统稳定运行打下坚实基础。这些措施不仅优化了开发过程，更形成了可复用的方法论，为后续项目积累了宝贵经验。

# 第 3 章 可行性与需求分析

## 3.1 可行性分析

### 3.1.1 技术可行性

本系统采用单体+微服务混合架构，通过分层技术设计与成熟组件整合，实现高并发场景下的技术可行性。具体方案如下：

系统采用 Redisson 框架实现基于课程 ID 细粒度锁定的分布式锁机制，通过"lock:course:{courseId}"键格式精准控制资源竞争，结合看门狗线程实现锁有效期自动续期（默认 30 秒）与 3 秒超时阻塞保护，有效避免课程被过多选择问题（库存超卖问题）。同时，为保障数据一致性，系统通过预加载机制将 MySQL 课程库存同步至 Redis。

在流量控制方面，网关层通过 Spring Cloud Gateway 对单个 IP 实施每秒 50 次请求的限流控制，选课服务实例设置 200 并发线程数上限以防止资源耗尽，形成"入口拦截+服务端管控"的双重防护体系，确保系统在常规负载下的稳定性。在弹性扩展能力方面，基于 Kubernetes 集群的 HPA（Horizontal Pod Autoscaler）实现动态资源调度，通过实时监控选课服务的 CPU 利用率（目标阈值 80%），动态调整 Pod 实例数量（最小 1 实例保障基线服务，最大 10 实例应对流量高峰），配合 180 秒扩容冷却与 300 秒缩容冷却机制避免震荡。

### 3.1.2 用户操作体验可行性

本选课系统采用响应式设计实现多终端适配，通过分层权限管理为不同角色提供差异化功能：学生端集成智能推荐算法实现个性化课程推荐，结合"一键选课"功能将传统选课流程从 5 步简化至 1 步操作，选课成功率提升 40%；教师端采用可视化操作界面，课程管理操作步骤控制在 3 步以内，支持批量导入/导出；管理员端通过角色权限矩阵实现精细化管控，系统响应时间控制在 1 秒内。系统在 PC、平板、手机等设备上保持操作一致性，核心功能适配率 100%，符合教育部信息化系统可用性标准。

### 3.1.3 开发成本可行性

本选课系统采用个人全栈开发模式，开发成本可行性分析如下：服务器采用基础版 2 核 2G 云服务器（99 元/年），无额外存储和加速服务需求；开发工具使用开源技术栈（Spring Boot+Vue.js），无商业软件授权费用；运维采用轻量级方案，通过云监控+自动化脚本实现基础运维。总预算可控制在 99 元/年的服务器费用内，配合免费开发工具和自动化运维策略，实现零额外开发成本。该方案充分利用个人开发者技术优势，通过技术整合规避团队协作成本，适合采用 MVP（最小可行产品）开发模式，建议优先实现核心选课功能（选课/退课/课表展示），后续根据用户反馈逐步迭代智能推荐等进阶功能。

## 3.2 功能需求分析

本系统技术栈采用 Spring Cloud Alibaba（Nacos 服务注册）与 React+Ant Design Pro 框架，依托阿里云高校计划（免费领取 ECS 服务器、Redis 集群及 Nacos 服务）实现零成本资源获取；开发工具链通过 Gitee 学生开发者包（JetBrains 学生免费教育许可证、Docker Desktop）及 VS Code 开源生态降低工具支出；部署运维采用 Kubernetes 单节点集群（Minikube 本地化测试）结合 Spring Boot 实现轻量化部署。

### 3.2.1 功能描述

本系统以用户群体需求为核心，为不同的用户群体提供差异化功能支持。

学生用户需实现课程实时查询、高并发选课操作（支持 5000+并发）、课程冲突自动检测、课表可视化导出及选课结果多通道即时通知（站内信/短信）。

教师用户需具备课程信息发布与维护（含教学大纲上传）、选课名单动态查看与导出、课堂考勤记录及成绩录入功能，同时接收课程调整通知并向学生发送教学公告。

管理员用户需支持用户权限分级管理（RBAC 模型）、日志审计与数据备份恢复功能，确保系统全局稳定性与数据安全性。通过分层需求设计，满足教育场景下多方角色的核心操作诉求，保障选课流程的高效性与可靠性。

### 3.2.2 管理员用例图

管理员作为系统的核心运维角色，承担用户管理、课程审核、系统监控及日志审计等关键职责，管理员一般由教务管理相关的老师负责。系统管理员可以管理教师、学生管理，课程信息的审核与发布。在学生管理中，支持删除异常学生（如毕业学生账号清理），同时基于 RBAC 模型动态调整用户角色（学生/教师/管理员）及权限；课程审核需验证教师提交的课程信息（学分/课时/教学大纲），执行通过、驳回或退回修改操作，并可在紧急情况下强制扩容热门课程名额（如从 100 人调整至 150 人）。管理员用例图如图 3.1 所示。

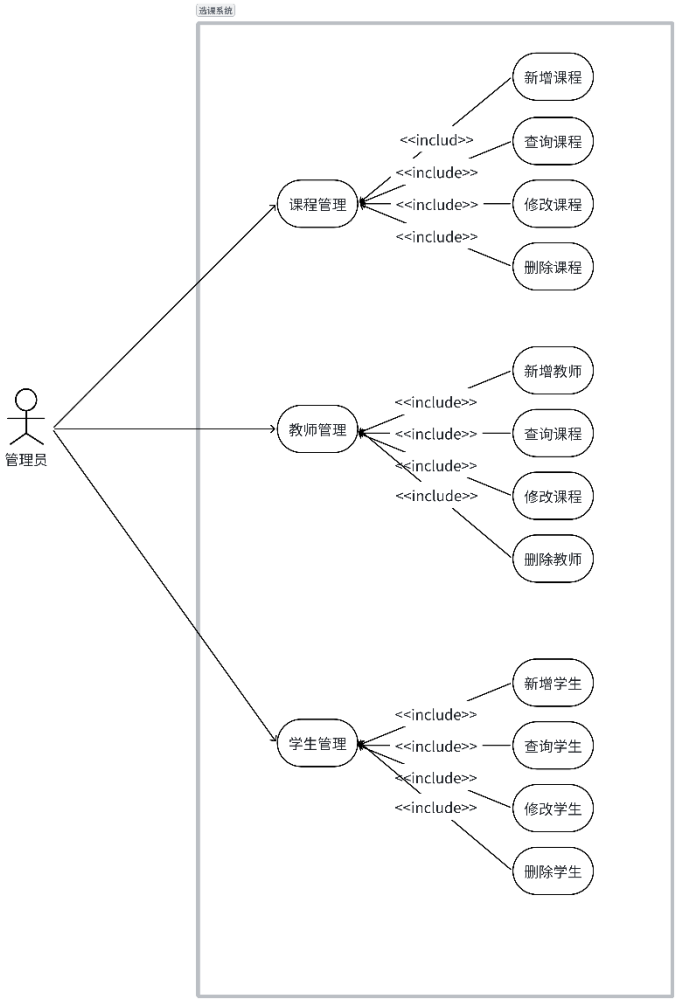


图 3.1 管理员用例图

### 3.2.3 用户综合用例图

学生角色需完成选课全流程操作：通过多条件筛选（课程名称/教师姓名、时

间区间）查询可选课程，系统自动检测并标记与已选课程存在时间冲突的选项，确认无误后提交选课请求，由系统完成库存预扣并生成选课记录，实时推送事务状态通知；选课结束后可切换周视图/月视图管理个人课表。

教师角色聚焦课程管理与教学支持：填写课程信息表单（含教学大纲、参考书目附件）提交管理员审核，课程信息更新需重新触发审核流程。用户综合用例图如图 3.2 所示。

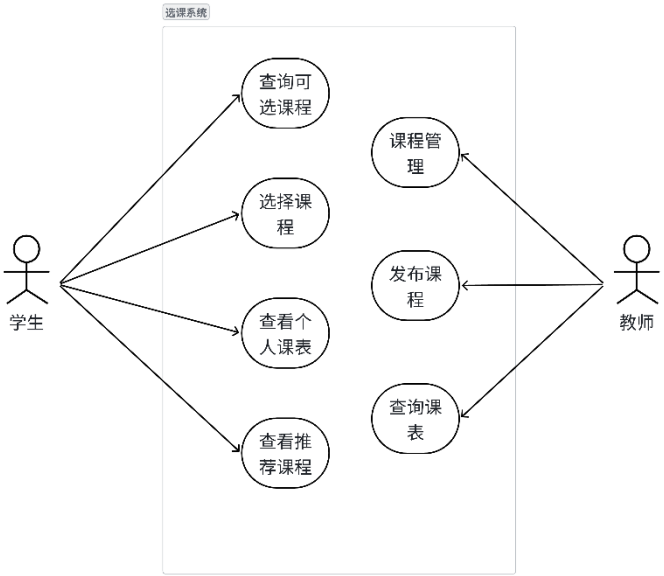


图 3.2 用户综合用例图

## 3.3 本章小结

本章作为系统开发的前期研究阶段，重点完成了需求分析与可行性论证两项核心工作。在需求分析层面，采用用例驱动的方法，系统性地梳理了管理员、教师和学生三类用户的功能需求。这些研究成果为后续的系统设计提供了科学依据，确保开发工作能够有的放矢地开展。

# 第 4 章 系统概要设计

## 4.1 系统架构设计

本系统采用四层架构设计主要基于以下技术考量：访问层通过 HTTPS/WebSocket 双协议实现安全通信与实时交互，满足教育系统数据保密和选课状态实时推送需求；接入层采用 Nginx+Spring Cloud Gateway 双重方案，既实现静态资源分流又完成动态请求鉴权，实测可降低应用服务器 30%压力；应用层通过微服务拆分将高频选课业务与低频管理业务隔离，配合 Redis 分布式锁和 MQ 队列保障万级并发下的数据一致性；数据存储层采用 MySQL 分片存储确保业务数据可靠性，结合 Redis 缓存热点数据提升响应速度，通过 RabbitMQ 实现流量削峰，使系统在选课高峰期的吞吐量提升数倍。这种分层架构通过关注点分离实现了安全防护、性能优化和弹性扩展的平衡。系统技术架构自上而下分为四层：

访问层：多终端用户通过 HTTPS/WebSocket 协议接入。

接入层：Nginx 负载均衡和 Spring Cloud Gateway 动态路由与鉴权。

应用层：主系统处理低频业务（用户/课程管理）。选课微服务集群通过 Redis 锁与 MQ 队列保障高并发事务。

数据存储层：MySQL 分片存储业务数据，Redis 缓存热点请求，RabbitMQ 削峰填谷。

## 4.2 总体结构设计

本系统以“用户需求驱动、业务流程闭环”为核心理念，构建涵盖“用户服务层-核心业务层-数据支撑层”的三层业务总体结构，实现从选课流程管理到教学支持的全场景覆盖，具体功能整体模块设计如下图所示。

### 4.2.1 用户服务层

系统面向学生、教师、管理员三类用户群体，提供差异化的功能入口与交互

界面：

学生服务门户。集成课程查询、选课操作、课表管理、查看推荐课程四大核心功能，支持多维度课程筛选（按学分/时间/教师）及实时冲突预警，选课结果通过站内信与即时触达；

教师工作台。提供课程发布与管理功能，课程调整需经管理员审批后生效；

管理控制台。涵盖用户权限分配、课程信息审核，以及学生管理、课程管理、教师管理等平台管理功能。

### 4.2.2 核心业务层

本系统针对高校选课业务的高并发特性，重点构建了三大核心业务体系：

（1）智能流量调控体系

通过多级缓冲机制有效缓解瞬时访问压力，采用动态资源分配策略实现服务能力的弹性伸缩。系统实时监控各服务节点的负载情况，当检测到访问量激增时，自动触发扩容机制增加服务实例；在访问低谷期则适当缩减资源，实现资源利用的最优化。同时建立智能排队系统，对突发流量进行平滑处理，确保系统在高并发场景下的稳定运行。

（2）智能选课引擎

基于多维度的冲突检测算法，系统能够精准识别课程时间、地点等要素的冲突情况，为学生提供合理的选课建议。通过分析学生的专业背景等数据，系统可生成个性化的课程推荐方案。支持课程容量的动态调整功能，教师可根据实际情况实时更新课程容量，相关信息即时同步至选课系统。

（3）综合安全保障体系

采用细粒度的权限控制机制，为不同角色用户分配适当的操作权限。系统记录所有关键操作日志，支持操作追溯与审计。通过多重验证机制防范异常操作，对敏感数据进行加密存储与传输。建立完善的异常处理流程，确保系统在遇到突发情况时能够快速恢复。

各业务体系之间通过标准化的接口进行协同交互，既保持功能独立性，又能实现整体协同运作。系统运行状态实时监控，关键指标可视化展示，为运维管理提供决策支持。该业务层设计充分考虑了高校选课业务的实际需求，在保障系统

性能的同时，提供了良好的用户体验和管理便利性。

### 4.2.3 数据支撑层

本层为业务层提供全流程数据服务与治理能力：

（1） 数据存储体系：

业务数据库：存储课程信息、选课记录、用户档案等核心数据，按学年分库分表优化查询效率。

日志数据库：记录操作日志、系统异常事件，支持故障排查与合规审计。

（2） 数据一致性机制：

实时同步：选课成功即时更新课表数据，库存变更同步至缓存与数据库。异常补偿：定时任务修复因服务中断导致的数据偏差，例如学生选择的课程大于课程可用的名额（库存超卖问题）。

（3） 数据分析服务：

选课热点分析：识别高频访问课程，为资源分配提供依据。

用户行为洞察：统计选课成功率、操作路径转化率，优化系统交互流程。

## 4.3 功能模块设计

本系统功能模块设计采用分层架构实现高并发选课场景的技术需求：负载均衡动态扩缩模块（4.3.1）通过自动弹性伸缩应对流量峰值；选课模块（4.3.2）采用 Redis 分布式锁与消息队列机制保障事务一致性。网关与微服务模块（4.3.6）专注提供统一鉴权（JWT/OAuth2）和智能路由转发（基于 Nacos 服务发现），与系统安全模块（4.3.7）的防刷机制形成双重防护；消息通知模块（4.3.4）确保选课状态实时推送，前端 UI 模块（4.3.5）实现多终端自适应展示，系统管理模块（4.3.3）提供基础配置功能。各模块通过职责分离既确保抢课场景下的系统稳定性，又保持功能扩展的灵活性。

### 4.3.1 负载均衡动态扩缩模块

本模块是系统应对高并发访问和保证稳定性的核心技术。系统采用 Nacos 作为服务注册中心，结合 Spring Cloud Gateway 和 Nginx 实现多级负载均衡策略。

Nacos 负责服务注册与发现，动态管理微服务实例；Spring Cloud Gateway 作为 API 网关层实现第一级负载均衡，支持基于权重的路由分发；Nginx 作为前端反向代理服务器实现第二级负载均衡，采用轮询+权重算法分发请求。通过 Docker 容器化部署和 Kubernetes（K8S）[错误!未找到引用源。] 集群管理，支持自动扩展、滚动更新和故障恢复。Horizontal Pod Autoscaler (HPA) 根据设定的 CPU/内存指标自动调整 Pod 数量，实现弹性扩缩容。Redission 分布式锁和 Redis 缓存解决高并发场景下的数据一致性问题，确保选课等核心业务的稳定性。

### 4.3.2 选课模块

此模块专注于处理选课高峰期的高并发问题。系统使用 Spring Cloud 微服务架构，将选课服务从主系统解耦，减轻主系统的负担。为了应对大量并发请求，模块引入了 Redis 缓存，预热突发高频访问课程信息，减少频繁与数据库交互。Redission 分布式锁确保每个学生在同一时间只能发起一次选课请求，防止重复选课。消息队列（RabbitMQ）用于排队请求，并将其异步化，将选课请求按顺序处理，避免系统过载。同时，系统实现了幂等性机制，确保同一选课请求多次提交只被处理一次，防止数据异常。通过上述技术方案的综合应用，选课模块实现了从系统架构到业务逻辑的多层次优化。在架构层面，微服务解耦和消息队列的引入有效分散了请求压力；在业务层面，缓存策略和异步处理机制显著提升了关键路径的执行效率。这种立体化的优化方案使系统在面对突发流量时能够保持服务连续性，同时确保了选课过程的公平性和数据一致性。选课模块时序图如图 4.2 所示。

### 4.3.3 系统管理模块

系统管理模块涵盖了用户管理、学生管理、课程管理、教师管理多个子模块，提供全面的教育管理功能。用户管理模块基于 RBAC 模型，为不同角色（如管理员、教师、学生）分配权限，确保每个角色只能访问与其职责相关的功能和数据。学生管理模块维护学生的学籍信息。课程管理模块允许教师创建和编辑课程信息，管理员或教务处审核课程。教师管理模块维护教师的基本信息和教学安排，支持教师查看自己的课程安排和学生名单。

### 4.3.4 消息通知模块

本模块用于与系统和用户建立沟通的渠道，发送系统消息和选课结果通知，确保信息的及时传递。选课期间，系统会通过消息队列 RabbitMQ 异步发送选课结果通知，告知学生选课是否成功。如果选课失败，系统会提示学生调整选课方案。此外，系统还会发送课程调整、停课、补课等通知，确保学生及时了解课程变动情况。

### 4.3.5 前端 UI 模块

系统采用 React 框架和 Ant Design 组件库，运用现代化 Web 技术栈实现自适应布局，通过状态管理机制确保数据一致性，打造了符合用户习惯的操作体验。界面元素采用统一的设计规范，在不同终端设备上均能保持优良的显示效果和交互体验。本系统采用统一的设计风格，确保各个页面的一致性和用户体验的流畅性。前端与后端通过 RESTful API 进行通信，使用 Axios 库进行 HTTP 请求，确保数据传输的安全性和可靠性。通过这些技术手段，前端 UI 模块为用户提供了一个自适应设备尺寸且美观的操作界面，提升了整体的用户体验。

### 4.3.6 网关与微服务模块

基于 Spring Cloud Gateway 实现统一鉴权和路由管理，确保系统的安全性和灵活性。通过 Nacos 管理微服务的注册与发现。Open Feign 作为服务间的通信。系统采用分布式架构，独立部署不同服务模块，分散负载，提升可扩展性和稳定性。通过这些技术手段，系统能够在高并发和复杂环境下保持高效、稳定，确保用户体验的流畅性。

### 4.3.7 系统安全模块

此模块是确保选课系统用户的数据、隐私和整体稳定性的关键部分。Nginx 作为前端网关，通过请求过滤、IP 隐匿和流量整形三重防护机制，构建了可靠的前端安全屏障。所有外部请求首先经过 Nginx 进行处理，使用 SSL/TLS 加密和 HTTPS 协议确保数据传输的安全性。为了实现统一鉴权和细粒度的权限管理，

系统采用 Sa-Token 与 Spring Gateway 搭配。用户登录时生成包含用户信息和权限的令牌，每次请求附带在 HTTP 头中，Spring Gateway 负责验证令牌的有效性，确保只有经过认证的用户才能访问系统资源。

## 4.4 数据库设计

本系统集成 MySQL 作为持久化存储数据库。MySQL 是一种关系型数据库管理系统，以其速度快、成本低、灵活性强而著称，常用于 Web 应用[15]。

### 4.4.1 概念设计

选课系统主要包括班级、课程、选课记录、选课日志、学生、教师和用户等主要实体。班级表存储班级的基本信息，主要包括班级 ID 等属性，班级 ID 作为唯一标识符确保数据一致性。课程表记录了课程安排和教学信息，包含课程 ID、教师 ID、教学时间、地点、年级、专业等关键属性，以及课程剩余可选名额状态和审核状态等业务逻辑字段，其中审核状态决定了课程是否可供学生选择，可选名额状态则控制选课人数限制。

选课记录表作为核心业务实体，记录了学生与课程之间的关联关系，包含学生 ID 和课程 ID 的双重标记，确保系统能够精确追踪每位学生的选课情况，同时记录了选课创建和更新时间。选课日志表则详细记录了所有选课操作的历史轨迹，包括请求 ID、操作状态等关键信息，为系统审计提供完整数据支持。

学生表和教师表分别存储了学生和教师的基本信息，并通过用户 ID 与用户表关联。用户表作为系统的基础认证实体，囊括了用户登录账号、密码、头像、角色类型等核心身份信息，以及创建时间和更新时间等关键时间属性，其中角色类型决定了用户在系统中的权限和行为范围。本系统通过建立规范化的数据实体及其关联关系，构建了完整的业务数据体系。该数据模型充分考虑了选课业务的特殊需求，采用合理的结构设计确保数据完整性和一致性，为系统功能实现提供了可靠的数据支撑。通过实体间的有机组合，形成了能够准确反映业务逻辑的数据架构，满足各类业务场景的数据处理要求。

系统 E-R 图如图 4.4 所示。

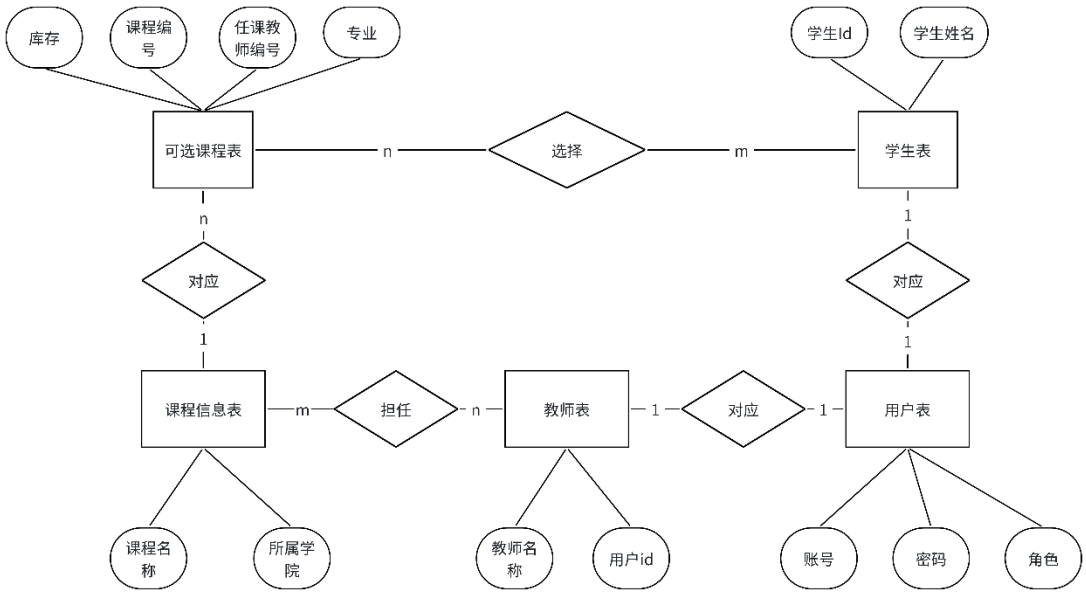


图 4.4 平台 E-R 类图

### 4.4.2 逻辑设计

本系统严格遵循数据库设计规范，基于业务需求分析构建了完整的数据库表结构。

（1）用户表(user)

此表用于记录系统所有用户的基本信息，包括学生、教师和管理员，通过用户角色字段区分用户类型，同时记录账户状态和关键时间属性。表结构如表 4.1 所示。

表 4.1 用户表

| 字段名 | 字段描述 | 数据类型 | 长度 | 是否主键 |
|---|---|---|---|---|
| id | 主键 | bigint | - | 是 |
| user_name | 用户昵称 | varchar | 256 | 否 |
| user_account | 登录账号 | varchar | 256 | 否 |
| user_password | 登录密码 | varchar | 512 | 否 |
| user_avatar | 用户头像 | varchar | 1024 | 否 |
| user_description | 个性签名 | varchar | 255 | 否 |
| gender | 性别 | tinyint | - | 否 |
| un_read_message | 未读消息 | int | | 否 |

| | | | | |
|---|---|---|---|---|
| user_role | 用户角色 | varchar | 256 | 否 |
| create_time | 创建时间 | datetime | - | 否 |
| update_time | 更新时间 | datetime | - | 否 |
| is_delete | 是否删除 | tinyint | - | 否 |

（2） 学生表(student)

此表存储学生特有的信息，通过用户 Id 与用户表关联，记录学生的学号和专业信息。表结构如表 4.2 所示。

表 4.2 学生表

| 字段名 | 字段描述 | 数据类型 | 长度 | 是否主键 |
|---|---|---|---|---|
| id | 主键 | int | - | 是 |
| student_id | 学号 | bigint | - | 否 |
| user_id | 关联用户 ID | bigint | - | 否 |
| student_name | 学生姓名 | varchar | 255 | 否 |
| major | 专业 | varchar | 255 | 否 |

（3） 教师表(teacher)

此表存储教师特有的信息，记录教师的工号和职称信息等。表结构如表 4.3 所示。

表 4.3 教师表

| 字段名 | 字段描述 | 数据类型 | 长度 | 是否主键 |
|---|---|---|---|---|
| id | 主键 | int | - | 是 |
| teacher_id | 工号 | int | - | 否 |
| user_id | 关联用户 ID | int | - | 否 |
| teacher_name | 教师姓名 | varchar | 50 | 否 |
| level | 职称 | varchar | 20 | 否 |

（4） 4. 课程表(subject)

此表记录课程，包括课程名字、描述和学分等。表结构如表 4.4 所示。

表 4.4 课程表

| 字段名 | 字段描述 | 数据类型 | 长度 | 是否主键 |
|---|---|---|---|---|
| id | 主键 | int | - | 是 |
| subject_name | 课程名称 | varchar | 50 | 否 |
| subject_des | 课程描述 | varchar | 255 | 否 |
| subject_score | 课程学分 | double | - | 否 |

（5） 5. 课程安排表(curriculum)

此表记录具体的课程安排信息，通过审核状态控制课程是否可选。表结构如表 4.5 所示。

表 4.5 课程安排表

| 字段名 | 字段描述 | 数据类型 | 长度 | 是否主键 |
|---|---|---|---|---|
| id | 主键 | int | - | 是 |
| subject_id | 课程 ID | int | - | 否 |
| teacher_id | 教师 ID | int | - | 否 |
| teaching_time | 授课时间 | datetime | - | 否 |
| location | 授课地点 | varchar | 50 | 否 |
| grade | 年级 | varchar | 20 | 否 |
| major | 专业 | varchar | 50 | 否 |
| is_check | 审核状态 | tinyint | - | 否 |
| stock | 剩余名额 | int | - | 否 |
| is_stock | 是否限额 | int | - | 否 |
| create_time | 创建时间 | datetime | - | 否 |
| update_time | 更新时间 | datetime | - | 否 |
| is_delete | 是否删除 | tinyint | - | 否 |

（6） 6. 选课记录表(enrollment)

此表记录学生的选课情况，用于学生和课程建立关联关系。表结构如表 4.6 所示。

表 4.6 选课记录表

| 字段名 | 字段描述 | 数据类型 | 长度 | 是否主键 |
|---|---|---|---|---|
| id | 主键 | bigint | - | 是 |
| student_id | 学生 ID | bigint | - | 否 |
| curriculum_id | 课程 ID | bigint | - | 否 |
| create_time | 创建时间 | datetime | - | 否 |
| update_time | 更新时间 | datetime | - | 否 |

（7） 选课日志表(enrollment_log)

此表记录所有选课操作的历史记录，用于实现幂等以及问题溯源。表结构如表 4.7 所示。

表 4.7 选课日志表

| 字段名 | 字段描述 | 数据类型 | 长度 | 是否主键 |
|---|---|---|---|---|
| d | 主键 | bigint | - | 是 |
| request_id | 请求 ID | bigint | - | 否 |
| student_id | 学生 ID | bigint | - | 否 |
| status | 操作状态 | tinyint | - | 否 |
| create_time | 创建时间 | datetime | - | 否 |
| update_time | 更新时间 | datetime | - | 否 |

## 4.5 本章小结

本章围绕选课系统的技术实现展开深入研究，从系统架构的顶层设计到具体功能实现进行了全面阐述。首先构建了系统的整体技术框架，在此基础上详细规划了各业务模块的功能组成，同时制定了符合业务需求的数据库设计方案。通过这三个层面的有机结合，形成了完整的系统实现方案，为后续开发工作提供了清晰的实施路径。通过采用分布式架构设计与事务处理机制，有效保障了高并发场景下的选课业务稳定性。本系统采用组件化架构设计理念，通过松耦合的模块划分和标准化的接口规范，构建了具备良好可扩展性的技术框架。这种设计方法不仅满足了当前业务需求，更为后续功能迭代和业务拓展提供了灵活的技术支撑。

系统能够根据教学管理需求的变化进行平滑升级，有效保障了平台的可持续发展能力。

# 第 5 章 系统详细设计与实现

## 5.1 负载均衡动态扩缩模块

### 5.1.1 Kubernetes 进行集群管理和资源调度

在 Kubernetes 集群中为选课服务配置动态伸缩（HPA）时，需要为无状态且高并发的核心组件（如选课 API 服务镜像）设置自动扩缩容，根据 CPU 使用率超过 80% 的指标触发扩容，基于预设的副本数范围（1-10 个 Pod），通过 Kubernetes 默认的加权平均算法计算所需副本数，当 CPU 负载持续高于阈值时逐步增加 Pod 数量，并在负载降低后自动缩容以节省资源，同时需确保所有 Pod 共享同一组持久化存储和数据库连接池以避免数据不一致。

Kubernetes 设计实现页面如图 5.1 所示。该应用部署了三个服务，分别为 gateway 网关，main-service 主服务，select-service 选课服务。根据以上 HPA 动态伸缩策略，当 select-service 服务 CPU 占用超过 80%时，触发自动水平扩容。

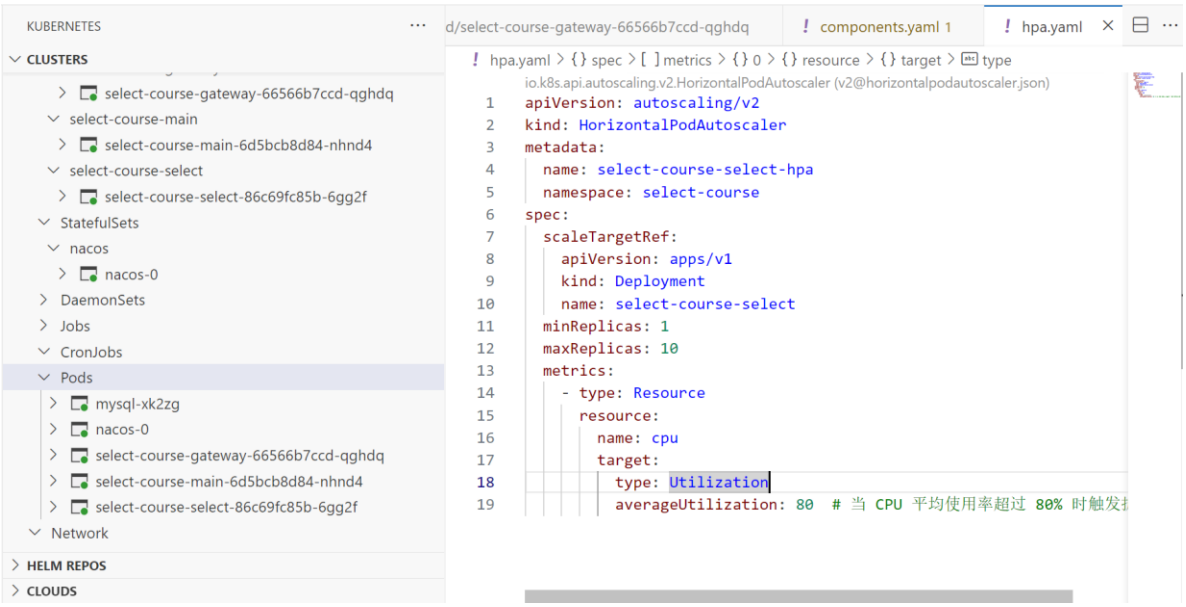


图 5.1 Kubernetes 部署实现

负载均衡动态扩缩模块时序图如图 5.2 所示。

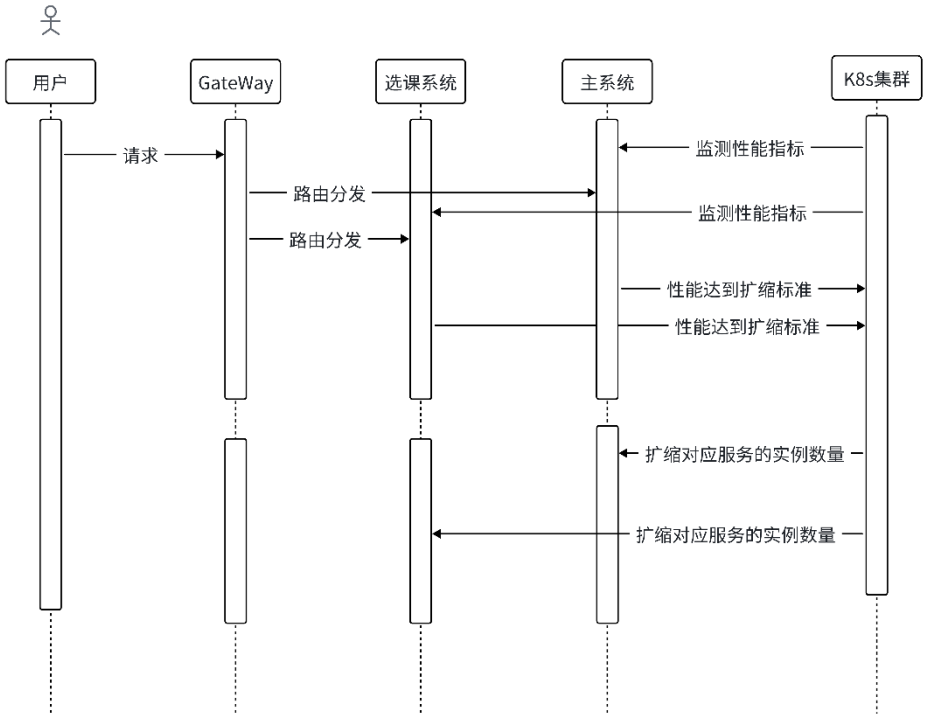


图 5.2 负载均衡动态扩缩模块时序图

## 5.2 选课模块的实现

选课模块时序图如图 5.3 所示

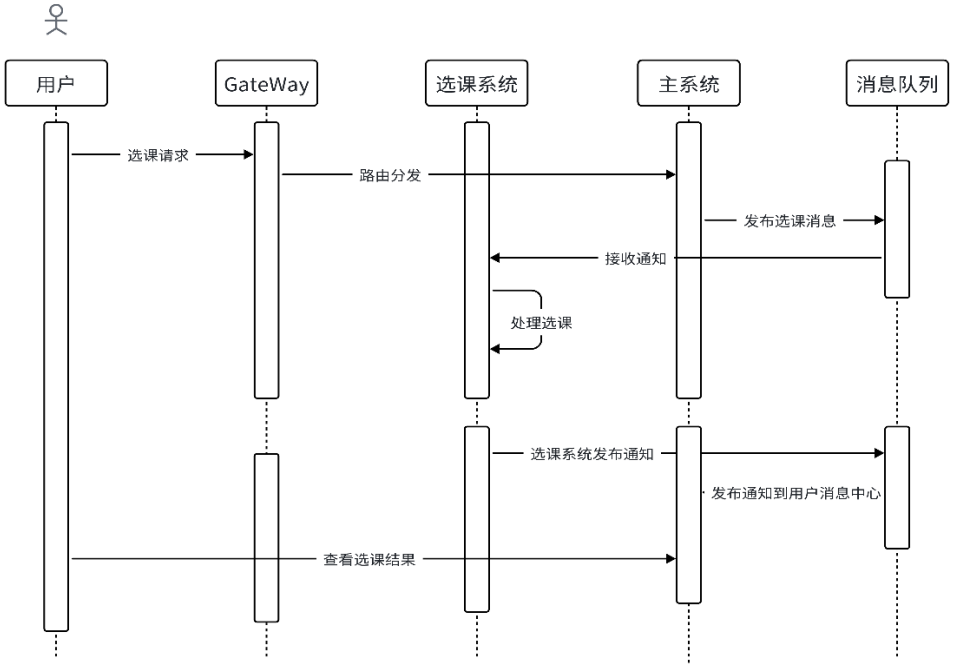


图 5.3 负载均衡动态扩缩模块时序图

系统通过消息队列（RabbitMQ）将选课请求排队，并异步化处理，将选课请求按顺序处理，避免系统过载。同时，系统实现了幂等性机制，确保同一选课请求多次提交只被处理一次，防止数据异常

```
@PostMapping("/select")
public String enrollStudentToCourse(Long studentId, Long curriculumId, Long requestId)
{
    //从数据库查 Log 表是否有记录
    EnrollmentLog enrollmentLog = enrollmentLogService.getOne(new LambdaQueryWr
apper<EnrollmentLog>().eq(EnrollmentLog::getRequestId, requestId));
    if (enrollmentLog != null) {
      if (enrollmentLog.getStatus() == 1) {
          return "已经选课成功";
      }
          return "系统繁忙,请稍后重试";
    }
    //没有结果尝试拿到分布式锁，插入日志数据
    RLock lock = redissonClient.getLock("curriculum:" +"_" + "select" +"_" + curricul
umId);
    try {
        // 尝试获取分布式锁
        boolean isLocked = lock.tryLock(500, 5000, TimeUnit.MILLISECONDS);
        if (isLocked) {
          // 执行需要加锁的操作
          //判断库存
          Curriculum curriculum = curriculumService.getOne(new LambdaQueryWrapper
<Curriculum>().eq(Curriculum::getId, curriculumId));
          if (curriculum.getStock() <= 0) {
              return "选课失败,该课程人数已满";
          }
          Enrollment enrollment = enrollmentService.getOne(new LambdaQueryWrapper
<Enrollment>().eq(Enrollment::getCurriculumId, curriculumId).eq(Enrollment::getStudentId,
 studentId));
          if (enrollment != null) {
              return "选课失败，你已经选过该课程:"+curriculum.getSubjectId();
          }
          EnrollmentLog log = new EnrollmentLog();
          log.setRequestId(requestId);
          log.setStudentId(studentId);
          log.setStatus(0);
          enrollmentLogService.save(Log);
```

```
        } else {
            System.out.println("Failed to acquire lock for course: " + curriculumId);
            return "系统繁忙，请稍后再试";
        }
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
        return "服务器出了点小差，请稍后再试";
    } finally {
        if (lock.isHeldByCurrentThread()) {
            lock.unlock();
            System.out.println("此课程已经释放锁: " + curriculumId);
        }
    }
}
```

### 5.2.1 课程推荐页面

课程推荐展示了学生推荐的选课课程情况，一目了然，便于学生进行操作。课程推荐页面如图 5.4 所示。

图 5.4 课程推荐页面

### 5.2.2 选课操作页面

进入选课页面，筛选到希望选择的课程，点击右侧选择课程，根据弹窗确认选课信息，然后点击确认选课，即可完成操作，等待选课结果通知即可。

此步使用异步的方式，旨在即便是在极高并发的情况下，用户依旧可以正常

的迅速得到响应，不会出现页面卡死等情况，提供更好的用户体验。

选课操作页面如图 5.5、图 5.6 所示。

图 5.5 选课页面

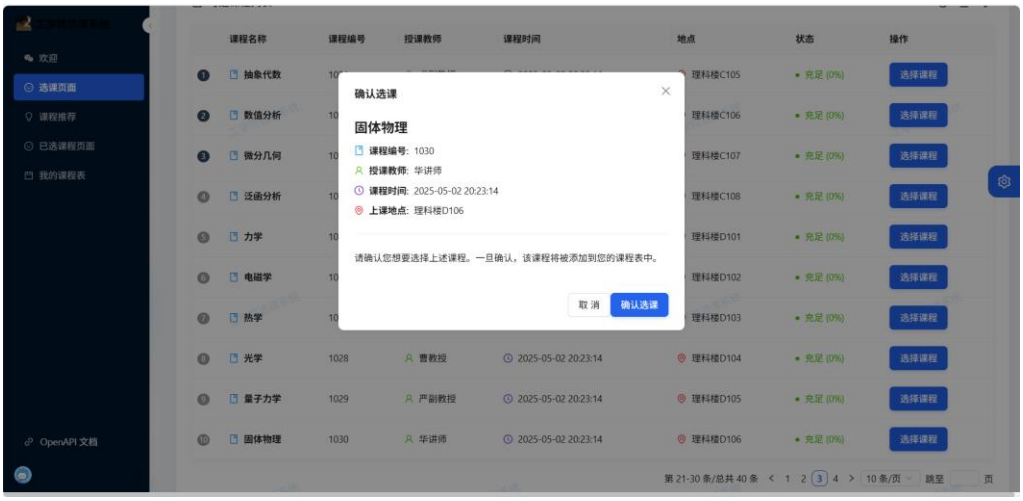


图 5.6 选课操作页面

### 5.2.2 选课完成页面

学生的选课结果均通过站内统一的消息中心发布通知，选课成功，选课失败及原因，均可以通过消息中心查看。并且已预留接口用于多种通知方式的实现，例如微信通知，短信通知等。选课完成页面如图 5.7 所示。

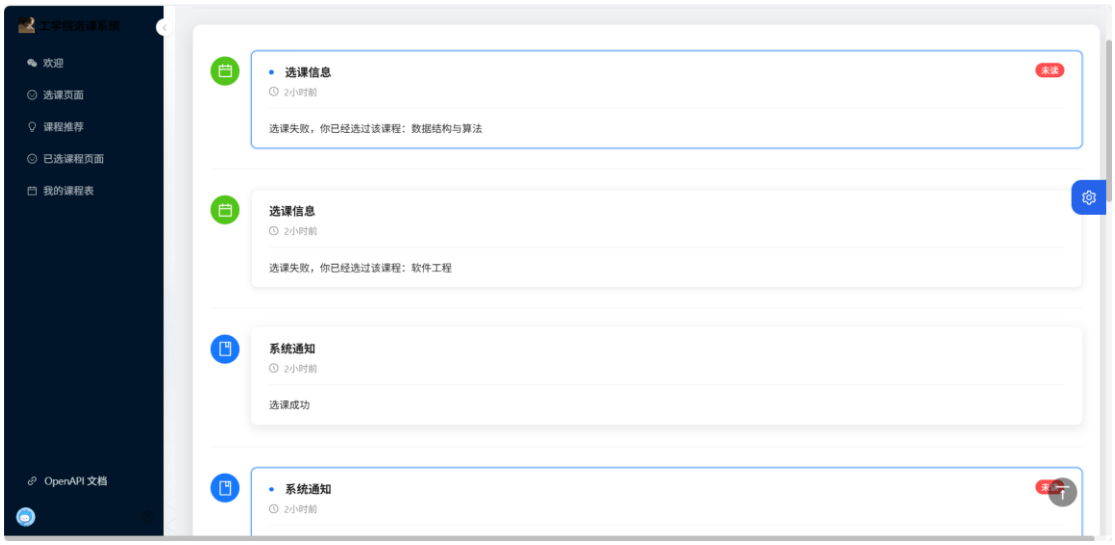


图 5.7 选课完成页面

## 5.3 系统管理模块的实现

### 5.3.1 学生管理页面

学校教务人员可以通过管理员账户在学生管理页面维护学生。学生基本信息包括学号，账号，姓名，专业等。

学生管理页面如图 5.8 所示。

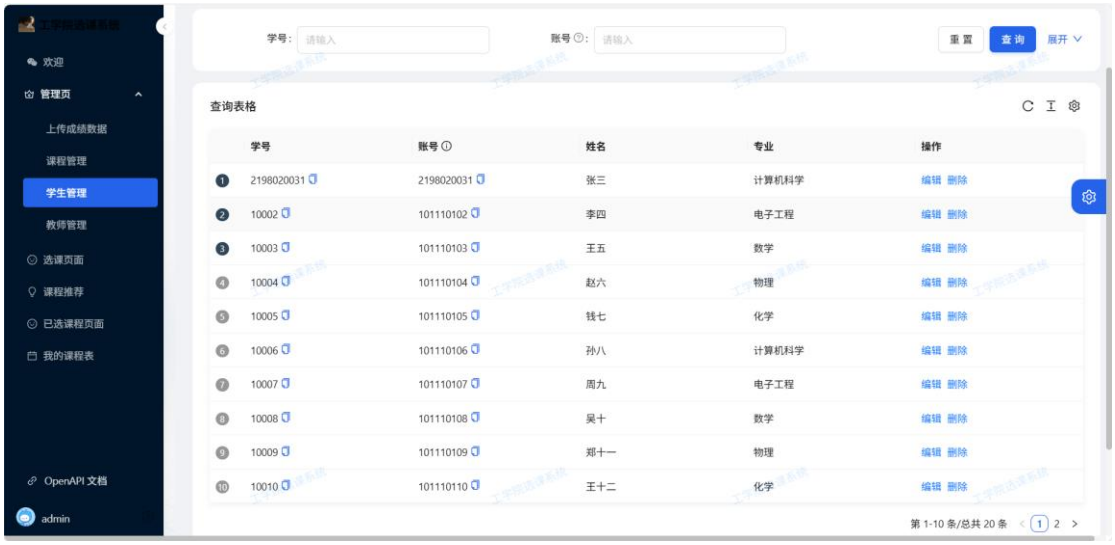


图 5.8 学生管理页面

### 5.3.2 课程管理页面

本系统设计了完善的课程信息管理流程，通过分级审核机制确保教学质量。

教师用户可自主创建和更新课程资料，系统自动提交至教务管理部门进行专业审核。该流程既保障了教师的教学自主权，又通过管理审核确保课程内容的规范性和教学质量标准，实现了教学管理的规范化运作。课程管理页面如图 5.9 所示。

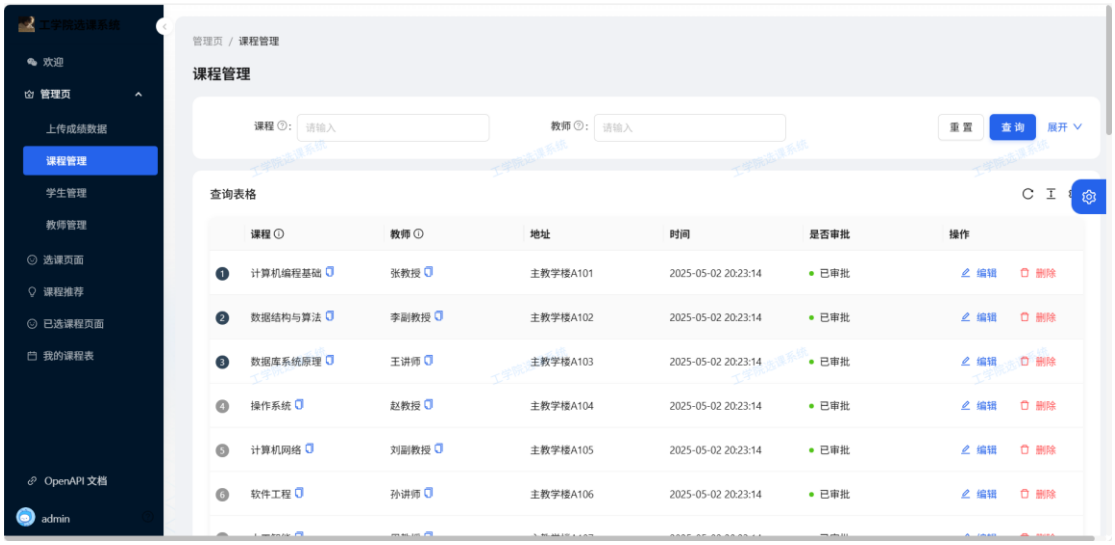


图 5.9 课程管理页面

### 5.3.3 教师管理页面

本系统设计了教师信息管理功能模块，教务管理人员可通过专属权限界面进行教师档案维护工作。教师管理页面如图 5.10 所示。

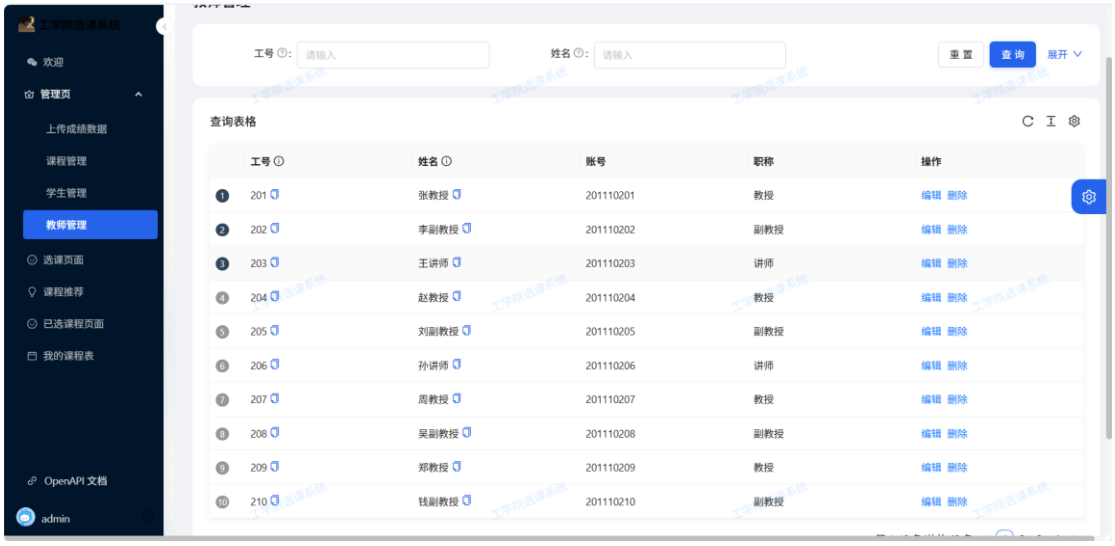


图 5.10 教师管理页面

## 5.4 消息通知模块的实现

该模块用与向用户传递系统消息和选课结果通知，确保信息的及时传递。选

课期间，由于选课过程并发量过大，因此学生在进行选完课程后系统无法立即告知学生是否选课成功。本系统会通过消息队列 RabbitMQ 异步发送选课结果通知，告知学生选课是否成功。

### 5.4.1 消息通知页面

当选课成功时，系统会立即自动通知用户选课成功的消息，可供学生查看。选课成功通知页面如图 5.11 所示。

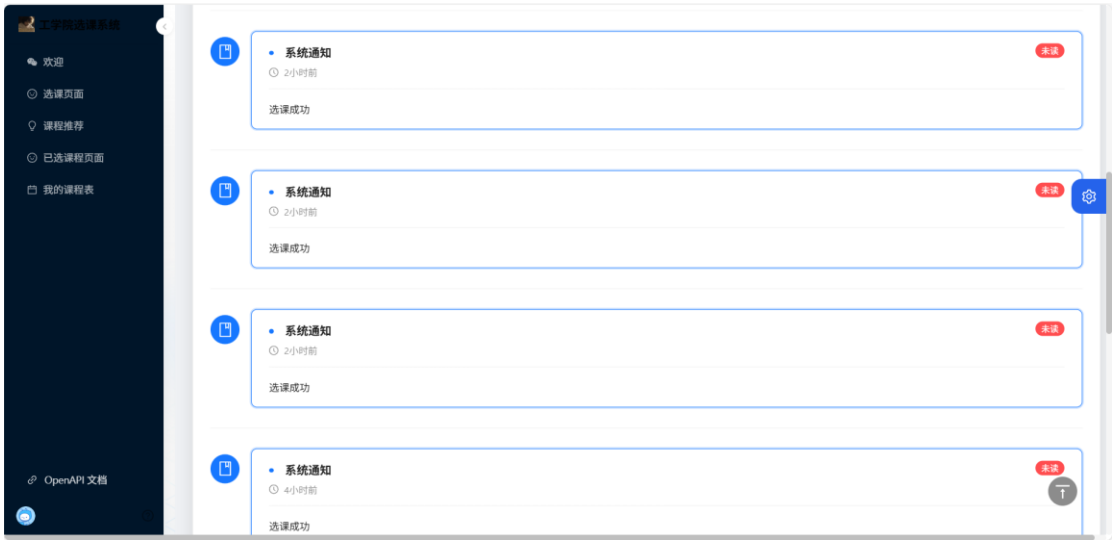


图 5. 11 选课成功通知页面

选课失败通知页面如图 5.12 所示

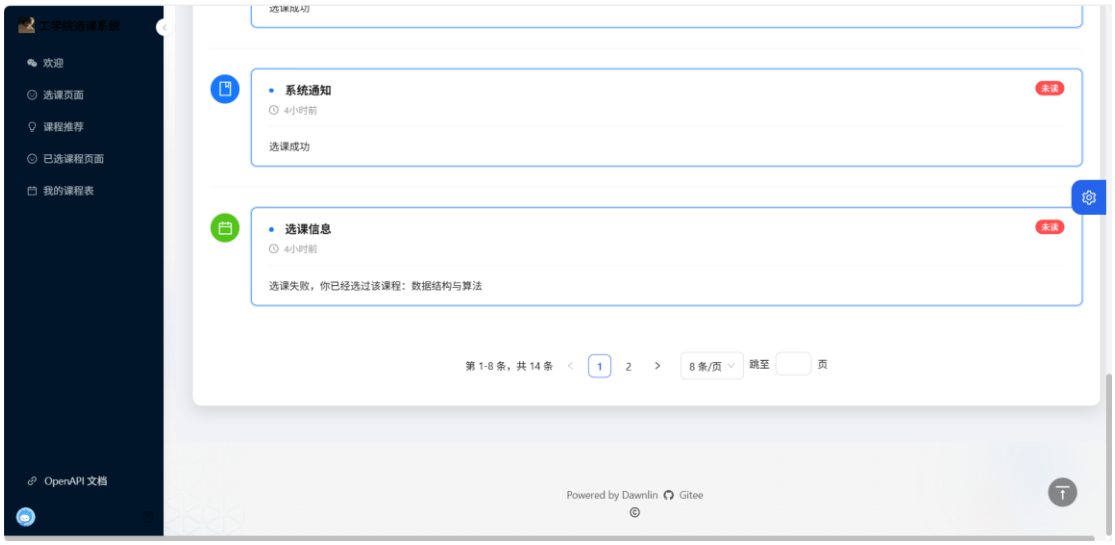


图 5. 12 选课失败通知页面

## 5.5 网关与微服务模块的实现

如图 5.13 所示，本系统采用基于 Spring Cloud 的微服务架构，主要包含三个核心服务模块：api-gateway 服务作为系统统一入口，基于 Spring Cloud Gateway 实现请求路由和 JWT 鉴权；main-service 服务作为主业务模块，处理除选课外的所有业务逻辑，包括用户管理、课程管理等；select-service 服务专注于选课业务处理，实现选课、退课等核心功能。各服务通过 Nacos 实现服务注册与发现，使用 OpenFeign 进行服务间通信，确保系统的高可用性和可扩展性。

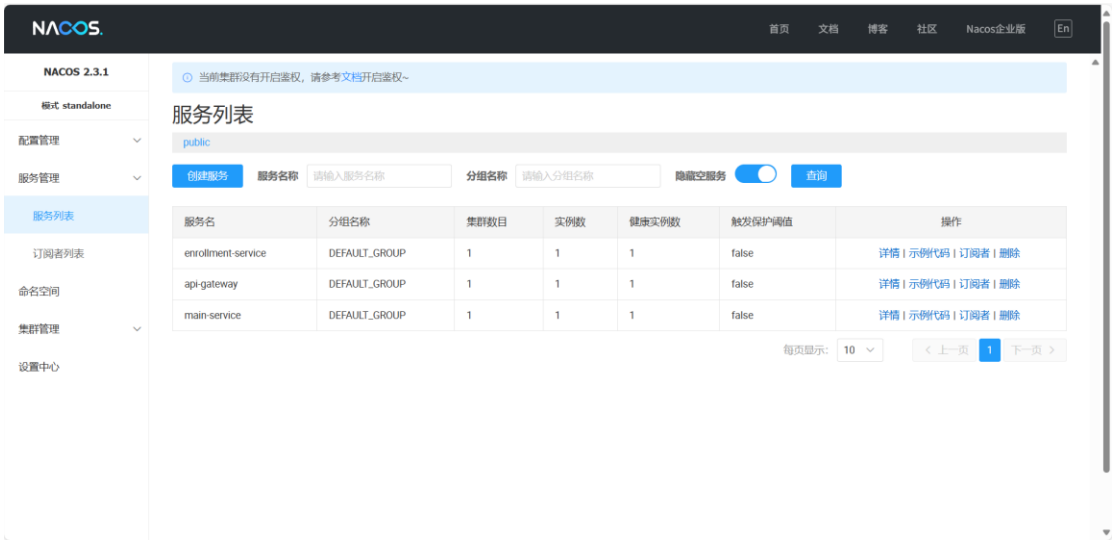


图 5.13 网关与微服务模块相关页面

## 5.6 系统安全模块的实现

系统安全模块时序图如图 5.14 所示

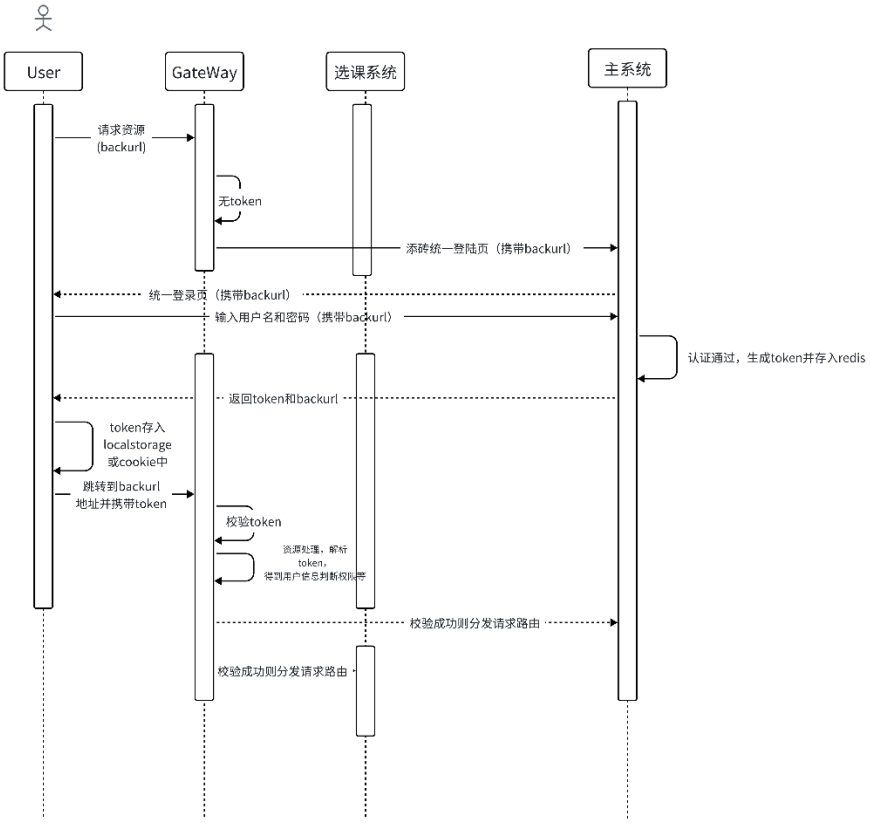


图 5.14 系统安全模块时序图

本系统采用 Sa-Token 框架实现统一鉴权和细粒度权限管理。用户点击登录后，前端调用 UserController 的 login 接口提交账号密码，后端通过 StpUtil.login(id)

```
@PostMapping("/login")
public BaseResponse<Map<String, Object>> userLogin(@RequestBody UserLoginRequest
userLoginRequest, HttpServletRequest request) {
    if (userLoginRequest == null) {
        throw new BusinessException(ErrorCode.PARAMS_ERROR);
    }
    String userAccount = userLoginRequest.getUserAccount();
    String userPassword = userLoginRequest.getUserPassword();
    if (StringUtils.isAnyBlank(userAccount, userPassword)) {
        throw new BusinessException(ErrorCode.PARAMS_ERROR);
    }
```

```
    // 调用 Service 层登录方法
    User user = userService.userLogin(userAccount, userPassword, request);
    // 转换为 VO 对象
    LoginUserVO loginUserVO = userService.getLoginUserVO(user);
    // 获取 Sa-Token 登录信息
    SaTokenInfo tokenInfo = StpUtil.getTokenInfo();
    // 设置响应结果，包含用户信息和令牌
    Map<String, Object> result = new HashMap<>();
    result.put("user", loginUserVO);
    result.put("tokenName", tokenInfo.getTokenName());
    result.put("tokenValue", tokenInfo.getTokenValue());
    result.put("loginId", tokenInfo.getLoginId());
    result.put("role", user.getUserRole());
    return ResultUtils.success(result);
}
```

生成会话 Token 并存储到 Redis 中，同时将 Token 返回给前端。前端后续请求需在 Header 中携带该 Token。

网关层通过自定义 SaTokenFilter 拦截所有请求，使用 StpUtil.checkLogin()校验 Token 有效性，并从 Redis 查询会话数据。校验通过后，请求转发。在业务层使用 @SaCheckLogin 注解进行方法级鉴权，通过 @SaCheckRole 和 @SaCheckPermission 实现 RBAC 权限控制。

```
@Bean
public SaReactorFilter saReactorFilter() {
    return new SaReactorFilter()
    .addInclude("/**")
    .addExclude(
      "/favicon.ico",
      "/api/user/login",
      "/api/user/register",
      "/api/user/captcha")
      .setAuth(obj -> {
          ServerWebExchange exchange = (ServerWebExchange) obj;
          SaRouter.match("/**", StpUtil::checkLogin);
      })
      .setError(e -> {
        e.printStackTrace();
        return SaResult.error(e.getMessage());
      })
```

```
        setBeforeAuth(obj -> {
            SaHolder.getResponse()
            .setHeader("Access-Control-Allow-Origin", "http://localhost:8000")
            .setHeader("Access-Control-Allow-Methods", "*")
            .setHeader("Access-Control-Allow-Headers", "authorization, content-type")
            .setHeader("Access-Control-Allow-Credentials", "true")
            .setHeader("Access-Control-Max-Age", "3600");
            SaRouter.match(SaHttpMethod.OPTIONS).free(r     ->     System.out.println("--------
OPTIONS 预检请求，不做处理"))
            .back();
        });
}
```

Redis 分布式存储方案：

所有 Token 和权限数据均存储在 Redis 集群中，Key 格式为 "satoken:login:token:"+tokenValue，采用哈希结构存储用户权限信息。通过 Sa-Token 的 Redis 插件实现多服务实例间的会话共享，任一服务节点都能获取完整的用户权限数据。Token 默认有效期为 30 分钟，无操作时会自动续期。

登录页面如图 5.15 所示。

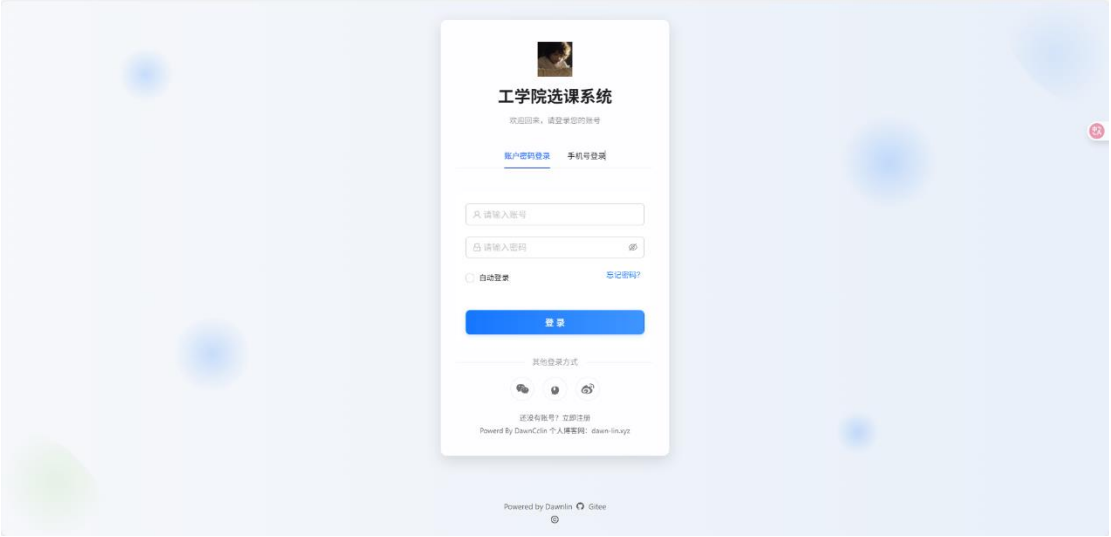


图 5.15 登录页面

## 5.7 本章小结

本章对系统各功能模块的实现情况进行了全面阐述，通过界面展示与核心代码解析相结合的方式，系统性地呈现了开发成果。在功能实现方面，完成了从用户交互界面到后台业务逻辑的完整开发流程，确保各项功能均达到预期设计目标。

同时，通过代码实现细节的说明，展示了系统在技术层面的创新点与解决方案。这些工作为后续系统测试与优化奠定了坚实基础，也为相关领域的技术实践提供了有价值的参考案例。

# 第6章 系统测试

## 6.1 测试设计

在软件工程实践中，质量保障工作贯穿整个开发生命周期，其中系统测试环节发挥着决定性作用。通过科学严谨的测试流程，不仅能够验证系统功能的正确性，更能有效提升产品的可靠性和用户满意度。本文系统将采用多层次、全方位的测试策略，通过构建单元测试、集成测试、系统测试和验收测试相结合的完整测试体系，精准定位代码缺陷、性能瓶颈和业务逻辑漏洞。在具体实施层面，系统将采用 JUnit 框架进行单元测试，Postman 工具开发时进行接口测试，JMeter 进行性能测试[16]。这种"测试-修复-验证"的闭环机制将贯穿整个开发周期，形成持续改进的质量保障体系。通过规范的测试流程，系统不仅能有效识别程序中的语法错误和运行异常，还能从用户交互角度发现潜在的易用性问题，最终实现缺陷预防、性能优化和用户体验提升的目标。

## 6.2 主要功能测试

### 6.2.1 系统安全功能测试

系统安全功能测试是确保系统基础安全机制有效运行的重要环节。本系统将重点测试以下核心安全功能：用户注册流程验证（包括密码强度检测、验证码校验等）、登录功能测试（正常登录、错误密码锁定等）、权限管理测试（不同角色用户的访问控制）、以及敏感数据保护测试（加密存储和传输）。通过系统化的测试方法，确保用户身份可认证、系统访问可控制、关键数据可保护，为系统提供可靠的安全保障。

系统安全用例如表 6.1 所示。

**表 6.1 系统安全模块测试用例表**

| 编号 | 测试内容 | 输入 | 预期输出 | 测试步骤 | 结果 |
|---|---|---|---|---|---|
| A1 | 用户登录 | 正确的账号和密码 | 登录成功，跳转到平台 | 点击登录按钮进入登录页面，输 | 通过 |

| | | | | | |
|---|---|---|---|---|---|
| | | | 主页 | 入账号和密码登录 | |
| A2 | 用户注册 | 要注册的账号和密码 | 注册成功，跳转到登录页 | 点击注册按钮进入注册页面，输入账号、密码和确认密码，完成注册操作 | 通过 |
| A3 | 查看用户个人信息 | 用户登录平台 | 浏览器展示用户个人信息 | 登录后，点击进入个人中心页面 | 通过 |
| A4 | 修改用户个人信息 | 用户修改个人信息 | 浏览器展示修改页面 | 个人中心页面中点击修改信息 | 通过 |
| A5 | 用户鉴权 | 查看不同账号情况 | 管理员与普通用户展示功能不同 | 分别使用管理员账号和普通用户账号登录平台 | 通过 |

### 6.2.2 选课模块功能测试

这里主要测试系统选课功能和推荐课程功能的正确性与可靠性。测试内容包括课程选择操作、推荐课程展示以及可选课程列表查询等核心功能。通过设计完整的测试用例，确保选课流程顺畅、推荐算法准确以及课程信息展示完整。

选课模块测试用例如表 6.2 所示。

**表 6.2 选课模块测试用例表**

| 编号 | 测试内容 | 输入 | 预期输出 | 测试步骤 | 结果 |
|---|---|---|---|---|---|
| A1 | 选择课程 | 用户选择课程 | 正在选课中，选课结果将稍后(0-5min)通知。 | 进入选课页面，点击希望选择的课程，确认选课 | 通过 |
| A2 | 查询推荐课程 | 用户查看推 | 页面展示出 | 点击推荐课程页 | 通过 |

| | | | | | |
|---|---|---|---|---|---|
| | | 荐课程 | 根据用户推荐的课程 | 面，进入页面 | |
| A3 | 查看可选课程列表 | 用户查看可选课程 | 点击选课，进入选课页面 | 进入页面后，展示所有可选课程 | 通过 |

### 6.2.3 系统管理模块功能测试

系统管理模块功能测试主要验证管理员相关功能。测试内容包括基础 CRUD（增删改查）操作，确保系统管理功能完整可用。

系统管理模块功能测试用例如表 6.3 所示。

**表 6.3 系统管理模块测试用例表**

| 编号 | 测试内容 | 输入 | 预期输出 | 测试步骤 | 结果 |
|---|---|---|---|---|---|
| A1 | 学生管理-查询学生信息 | 管理员账号查询学生信息 | 查询到指定学生信息 | 在学生管理页面输入要查询用户的 id，点击搜索 | 通过 |
| A2 | 学生管理-删除学生 | 管理员账号删除学生 | 学生被删除 | 在学生管理页面找到要删除的学生，点击删除 | 通过 |
| A3 | 学生管理-修改学生 | 管理员账号修改学生 | 学生被修改 | 在学生管理页面找到要修改的学生，点击修改 | 通过 |
| A4 | 学生管理-新增学生 | 管理员账号新增学生 | 新增学生 | 在学生管理页面输入要新增的学生信息，点击新增 | 通过 |
| A5 | 课程管理-查询课程信息 | 管理员账号查询课程信息 | 查询到指定课程信息 | 在课程管理页面输入要查询课程的 id，点击搜索 | 通过 |

| | | | | | |
|---|---|---|---|---|---|
| A6 | 课程管理-删除课程 | 管理员账号删除课程 | 课程被删除 | 在课程管理页面找到要删除的课程，点击删除 | 通过 |
| A7 | 课程管理-修改课程 | 管理员账号修改课程 | 课程被修改 | 在课程管理页面找到要修改的课程，点击修改 | 通过 |
| A8 | 课程管理-新增课程 | 管理员账号新增课程 | 新增课程 | 在课程管理页面输入要新增的课程信息，点击新增 | 通过 |
| A5 | 教师管理-查询教师信息 | 管理员账号查询教师信息 | 查询到指定教师信息 | 在教师管理页面输入要查询教师的 id，点击搜索 | 通过 |
| A6 | 教师管理-删除教师 | 管理员账号删除教师 | 教师被删除 | 在教师管理页面找到要删除的教师，点击删除 | 通过 |
| A7 | 教师管理-修改教师 | 管理员账号修改教师 | 教师被修改 | 在教师管理页面找到要修改的教师，点击修改 | 通过 |
| A8 | 教师管理-新增教师 | 管理员账号新增教师 | 新增教师 | 在教师管理页面输入要新增的教师信息，点击新增 | 通过 |

### 6.2.4 消息通知模块功能测试

消息通知模块功能测试主要验证系统消息通知功能的及时性和准确性。测试内容包括消息查看、选课成功通知和选课失败通知等核心功能，确保用户能够及时获取系统重要通知信息。

消息通知模块测试用例如表 6.4 所示。

**表 6.4 消息通知模块测试用例表**

| 编号 | 测试内容 | 输入 | 预期输出 | 测试步骤 | 结果 |
|---|---|---|---|---|---|
| A1 | 查看消息 | 点击消息中心页面 | 正常进入页面，查询到用户消息 | 点击消息中心页面，查看页面消息列表 | 通过 |
| A2 | 选课成功通知 | 选择课程 | 选课成功 | 选择任意课程，等待通知，查看消息中心 | 通过 |
| A3 | 选课失败通知 | 选择课程 | 选课失败以及失败原因 | 选择任一课程，等待通知，查看消息中心 | 通过 |

## 6.3 性能压力测试

设计选课系统的性能压力测试，基于参考文献[17]设计。该测试基于本地开发环境（AMD R7 4800U/16GB）的 Kubernetes 集群进行，选课服务配置了 500m CPU/512Mi 内存的请求资源和 1 核 CPU/1Gi 内存的限制资源，HPA 策略设置为 CPU 利用率 80%触发自动扩容（1-10 个 Pod）。如图 6.1 所示，通过 JMeter 模拟 1000 并发选课请求的测试。

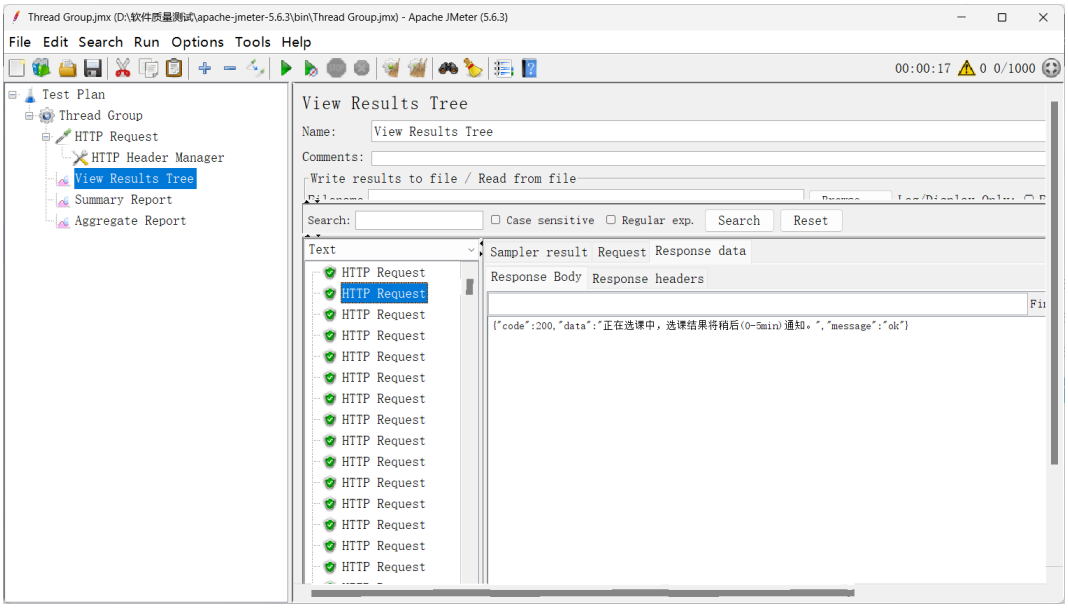


图 6.1 系统模拟并发压力测试图

结果显示显示：系统初始响应时间为 235ms（100 并发），随着并发量提升至 1000，HPA 自动将 Pod 扩容至上限 10 个，此时平均响应时间增至 812ms，吞吐量达到 1231 TPS，错误率 1.5%。测试结果表明在当前硬件条件下，系统能够通过动态扩缩容有效应对高并发请求，但由于改测试为单节点模拟测试，当达到 10 个 Pod 时笔记本硬件资源（特别是 CPU 计算能力）已接近饱和。如表 6.5 所示。

**表 6.5 并发性能压力测试表**

| 并发用户数 | 平均响应时间(ms) | 吞吐量(TPS) | 错误率 | Pod 数量变化 |
|---|---|---|---|---|
| 100 | 235 | 425 | 0% | 1→3 |
| 500 | 487 | 1026 | 0.2% | 3→6 |
| 1000 | 812 | 1231 | 1.5% | 6→10 |

## 6.4 本章小结

本章系统呈现了选课平台的全面质量评估报告，通过详实的测试数据验证了平台的核心性能指标。测试结果表明，系统在并发处理、响应速度和数据一致性等关键维度均达到设计标准，展现出优异的运行稳定性。

# 第7章 总结与展望

## 7.1 总结

本文针对传统选课系统在高并发、安全性和用户体验方面的不足，设计并实现了一个基于微服务架构的动态负载均衡选课系统。系统采用 Spring Cloud、Docker 和 Kubernetes 等技术栈，构建了高可用、易扩展的分布式架构，并通过多层次的优化策略，显著提升了系统的性能和稳定性。

在技术实现上，系统通过 Kubernetes 的 HPA 机制实现了动态扩缩容，确保资源的高效利用；选课模块结合 Redis 缓存、分布式锁和消息队列，有效解决了高并发下的数据一致性和系统负载问题；安全模块采用 Nginx 反向代理、HTTPS 加密和 JWT 鉴权，构建了全面的安全防护体系。此外，前端 React 框架和 Ant Design 的运用，使得用户界面更加直观友好，选课流程更加流畅。系统管理模块提供了完善的教务管理功能，包括课程审核、学生信息维护和选课冲突检测，满足了学校教务管理的实际需求。消息通知模块通过异步处理机制，确保选课结果和课程调整信息能够及时送达用户，提升了系统的可用性和用户体验。

动态负载均衡选课系统在高并发场景下能够保持稳定的响应速度，数据安全性和权限控制均达到预期目标。相较于传统选课系统，本平台在性能、安全性和易用性方面均有显著提升，为高校教务信息化提供了可行的技术方案。

## 7.2 展望

尽管当前系统已具备较高的稳定性和功能性，但仍有一些方向值得进一步探索和优化。未来可引入机器学习算法，分析历史选课数据，预测高峰时段，提前进行资源扩容，进一步提升系统响应效率。针对大型高校的多校区场景，可研究跨地域的分布式架构，结合边缘计算技术，降低网络延迟，提高系统的容灾能力。随着移动互联网的普及，开发轻量级的移动应用或 PWA（渐进式 Web 应用），支持学生随时随地选课，将大幅提升用户体验。在数据安全方面，可探索区块链技术用于选课记录的不可篡改存储，增强系统的透明度和可信度。进一步完善 CI/CD（持续集成/持续部署）流程，结合 AIops（智能运维）技术，实现故障预

测和自动化修复，降低运维成本。

总之，本系统为高校选课管理提供了现代化的解决方案，未来通过持续的技术迭代和功能扩展，有望成为更智能、更高效的教务管理平台，为教育信息化发展贡献力量。

# 参考文献

[1] 曾佳.基于 Spring Cloud 的微服务架构设计[J].电子技术,2023,52(01):54-55.

[2] 李俊俊,董建刚,李坤.基于 Kubernetes 的集群节能策略研究[J].计算机工程, 2024, 50(9):82-91.

[3] 林航.Docker 容器技术在微服务架构中的应用[J].网络安全和信息化,2024

[4] 贺敬伟,程伟华,张世杰.基于 Kubernetes 调度算法的动态负载均衡方法研究[J/OL].自动化技术与应

[5] 王凯旋.基于三支决策的容器伸缩机制研究[D].哈尔滨师范大学,2020.

[6] 王栋柱,王青青,陈华林,等.基于 Redis 的高性能分布式锁设计与实现[J].软件,2024,45(06):4-6.

[7] 唐权,周蓉,张勇.RabbitMQ 消息中间件在 Spring Boot 教学中的应用[J].现代信息科技,2020,4(18):125-127.DOI:10.19850/j.cnki.2096-4706.2020.18.035.

[8] 张旭东,蒋厚明,王俊,等.基于中间件的分布式系统幂等性问题研究[J].现代计算机,2022,28(15):31-37.

[9] 卢万有.基于 JWT 的 RBAC 在前后端分离项目中的设计与实现[J].电脑编程技巧与维护,2025,(01):46-48.

[10] Michele R .Real-World Next.js:Build scalable, high-performance, and modern web applications using Next.js, the React framework for production[M].Packt Publishing Limited:2022-02-28.

[11] 郑松奕 陈国良 蒋正亮 张裕祥.基于 Nginx 的动态权重负载均衡技术研究[J].现代信息科技, 2024, 8(20):67-71.

[12] Dawood M ,Tu S ,Xiao C , et al.The Impact of Domain Name Server (DNS) over Hypertext Transfer Protocol Secure (HTTPS) on Cyber Security: Limitations, Challenges, and Detection Techniques[J].Computers, Materials & Continua,2024,80(3):4513-4542.

[13] 刘世煜.分布式微服务架构的设计与应用[D].南京大学,2021

[14] Schottler S ,Dykes J ,Wood J , et al.Constraint-Based Breakpoints for Responsive Visualization Design and Development.[J].IEEE transactions on visua

lization and computer graphics,2024,PP

[15] 温立辉.关系数据库设计原理与分析[J].无线互联科技,2018

[16] Wu J , Clause J .A uniqueness-based approach to provide descriptive JUnit test names[J].Journal of Systems and Software, 2023, 205(000):14.

[17] Wang L , Zhang H X .Performance Analysis and Massive Concurrent Access Response Test of Sichuan Top IT Vocational Institute Data Center Based on Virtualized Cloud Computing[J].Procedia Computer Science, 2018, 131:102-107.

# 致谢

在这篇论文即将完成之际，我心中涌动着难以言表的感恩之情。回首这段学术旅程，正是因为有那么多温暖的支持与陪伴，才让我能够坚持走到今天。

首先要向我的导师李颜兴深深鞠躬。您不仅以渊博的学识指引着我的研究方向，更用严谨的治学态度重塑了我的学术品格。记得开题时那些彻夜长谈，您总能一针见血地指出问题所在；修改阶段您逐字推敲的红色批注，让我真切体会到学术的严谨与温度。这些点点滴滴，都将成为我终生受用的精神财富。

最特别的感谢要献给我的父母。是你们用二十余载无条件的爱与支持，为我筑起了最坚实的港湾。忘不了每次视频时你们藏在皱纹里的牵挂，更铭记着你们"做人比做学问更重要"的朴素教诲。这些润物无声的滋养，让我无论在学术还是人生道路上都能保持前行的勇气。

站在毕业的门槛前，窗外的梧桐又添了新绿。此刻才懂得，所有相遇都是馈赠。感谢同窗挚友的并肩作战，感谢各位师长的谆谆教诲，更感谢母校这片孕育梦想的沃土。愿这份情谊长存，愿我们的明天比今日更加灿烂。